

CSS3 Introduction

CSS3 is the latest standard for CSS.

CSS3 is completely backwards-compatible with earlier versions of CSS.

This section teaches you about the new features in CSS3!

CSS3 Modules

CSS3 has been split into "modules". It contains the "old CSS specification" (which has been split into smaller pieces). In addition, new modules are added.

Some of the most important CSS3 modules are:

- Selectors
- Box Model
- Backgrounds and Borders
- Image Values and Replaced Content
- Text Effects
- 2D/3D Transformations
- Animations
- Multiple Column Layout
- User Interface

Most of the new CSS3 properties are implemented in modern browsers.

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit- or -moz- specify the first version that worked with a prefix.

Property			
border-radius	5.0 4.0 -webkit-	9.0	4.0 3.0 -moz-

CSS3 border-radius Property

With CSS3, you can give any element "rounded corners", by using the **border-radius** property.

Here are three examples:

1. Rounded corners for an element with a specified background color:

Rounded corners!

2. Rounded corners for an element with a border:

Rounded corners!

3. Rounded corners for an element with a background image:

Rounded corners!

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#rcorners1 {
```

```
    border-radius: 25px;
```

```
    background: #73AD21;
```

```
    padding: 20px;
```

```
    width: 200px;
```

```
    height: 150px;
```

```
}
```

```
#rcorners2 {
```

```
    border-radius: 25px;
```

```
    border: 2px solid #73AD21;
```

```
    padding: 20px;
```

```
    width: 200px;
```

```
    height: 150px;
```

```
}
```

```
#rcorners3 {
```

```
border-radius: 25px;

background: url(paper.gif);

background-position: left top;

background-repeat: repeat;

padding: 20px;

width: 200px;

height: 150px;

}

</style>

</head>

<body>
```

<p>The border-radius property allows you to add rounded corners to elements.</p>

<p>Rounded corners for an element with a specified background color:</p>

<p id="rcorners1">Rounded corners!</p>

<p>Rounded corners for an element with a border:</p>

<p id="rcorners2">Rounded corners!</p>

<p>Rounded corners for an element with a background image:</p>

<p id="rcorners3">Rounded corners!</p>

</body>

</html>

Tip: The `border-radius` property is actually a shorthand property for the `border-top-left-radius`, `border-top-right-radius`, `border-bottom-right-radius` and `border-bottom-left-radius` properties.

CSS3 border-radius - Specify Each Corner

If you specify only one value for the `border-radius` property, this radius will be applied to all 4 corners.

However, you can specify each corner separately if you wish. Here are the rules:

- **Four values:** first value applies to top-left, second value applies to top-right, third value applies to bottom-right, and fourth value applies to bottom-left corner
- **Three values:** first value applies to top-left, second value applies to top-right and bottom-left, and third value applies to bottom-right
- **Two values:** first value applies to top-left and bottom-right corner, and the second value applies to top-right and bottom-left corner
- **One value:** all four corners are rounded equally

Here are three examples:

1. Four values - `border-radius: 15px 50px 30px 5px;`
2. Three values - `border-radius: 15px 50px 30px;`
3. Two values - `border-radius: 15px 50px;`

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#rcorners4 {
```

```
    border-radius: 15px 50px 30px 5px;
```

```
    background: #73AD21;
```

```
    padding: 20px;
```

```
    width: 200px;
```

```
    height: 150px;
```

```
}
```

```
#rcorners5 {
```

```
    border-radius: 15px 50px 30px;
```

```
    background: #73AD21;
```

```
    padding: 20px;
```

```
    width: 200px;
```

```
    height: 150px;
```

```
}
```

```
#rcorners6 {
```

```
    border-radius: 15px 50px;
```

```
    background: #73AD21;
```

```
padding: 20px;

width: 200px;

height: 150px;
}

</style>

</head>

<body>

<p>Four values - border-radius: 15px 50px 30px 5px:</p>

<p id="rcorners4"></p>

<p>Three values - border-radius: 15px 50px 30px:</p>

<p id="rcorners5"></p>

<p>Two values - border-radius: 15px 50px:</p>

<p id="rcorners6"></p>

</body>

</html>
```

You could also create elliptical corners:

Example

```
<!DOCTYPE html>
```

```
<html>

<head>

<style>

#rcorners7 {

    border-radius: 50px/15px;

    background: #73AD21;

    padding: 20px;

    width: 200px;

    height: 150px;

}
```

```
#rcorners8 {

    border-radius: 15px/50px;

    background: #73AD21;

    padding: 20px;

    width: 200px;

    height: 150px;

}
```

```
#rcorners9 {

    border-radius: 50%;

    background: #73AD21;
```


padding: 20px;

Property	Description
width: 200px;	
height: 150px;	
}	
</style>	
</head>	
<body>	
Elliptical border - border-radius: 50px/15px:	
<p id="rcorners7"></p>	
Elliptical border - border-radius: 15px/50px:	
<p id="rcorners8"></p>	
Ellipse border - border-radius: 50%:	
<p id="rcorners9"></p>	
</body>	
</html>	

CSS3 Rounded Corners Properties

border-radius	A shorthand property for setting all the four border-*-*radius properties
border-top-left-radius	Defines the shape of the border of the top-left corner
border-top-right-radius	Defines the shape of the border of the top-right corner
border-bottom-right-radius	Defines the shape of the border of the bottom-right corner
border-bottom-left-radius	Defines the shape of the border of the bottom-left corner

CSS3 Border Images

With the CSS3 **border-image** property, you can set an image to be used as the border around an element.

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit-, -moz-, or -o- specify the first version that worked with a prefix.

Property			
----------	--	--	--

border-image	16.0 4.0 -webkit-	11.0	15.0 3.5 -moz-
--------------	----------------------	------	-------------------

CSS3 border-image Property

The CSS3 `border-image` property allows you to specify an image to be used instead of the normal border around an element.

The property has three parts:

1. The image to use as the border
2. Where to slice the image
3. Define whether the middle sections should be repeated or stretched

We will use the following image (called "border.png"):



The `border-image` property takes the image and slices it into nine sections, like a tic-tac-toe board. It then places the corners at the corners, and the middle sections are repeated or stretched as you specify.

Note: For `border-image` to work, the element also needs the `border` property set!

Here, the middle sections of the image are repeated to create the border:

An image as a border!

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#borderimg {
```

```
    border: 10px solid transparent;
```

```
    padding: 15px;
```

```
    -webkit-border-image: url(border.png) 30 round; /* Safari 3.1-5 */
```

```
    -o-border-image: url(border.png) 30 round; /* Opera 11-12.1 */
```

```
    border-image: url(border.png) 30 round;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>The border-image property specifies an image to be used as the border around an element:</p>

<p id="borderimg">Here, the middle sections of the image are repeated to create the border.</p>

<p>Here is the original image:</p>

<p>Note: Internet Explorer 10, and earlier versions, do not support the border-image property.</p>

</body>

</html>

Here, the middle sections of the image are stretched to create the border:

An image as a border!

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#borderimg {
```

```
    border: 10px solid transparent;
```

```
    padding: 15px;
```

```
    -webkit-border-image: url(border.png) 30 stretch; /* Safari 3.1-5 */
```

```
    -o-border-image: url(border.png) 30 stretch; /* Opera 11-12.1 */
```

```
    border-image: url(border.png) 30 stretch;
```

```
}
```

```
</style>
```

</head>

<body>

<p>The border-image property specifies an image to be used as the border around an element:</p>

<p id="borderimg">Here, the middle sections of the image are stretched to create the border.</p>

<p>Here is the original image:</p>

<p>Note: Internet Explorer 10, and earlier versions, do not support the border-image property.</p>

</body>

</html>

Tip: The `border-image` property is actually a shorthand property for the `border-image-source`, `border-image-slice`, `border-image-width`, `border-image-outset` and `border-image-repeat` properties.

CSS3 border-image - Different Slice Values

Different slice values completely changes the look of the border:

Example 1:

```
border-image: url(border.png) 50 round;
```

Example 2:

border-image: url(border.png) 20% round;

Example 3:

border-image: url(border.png) 30% round;

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#borderimg1 {
```

```
border: 10px solid transparent;
```

```
padding: 15px;
```

```
-webkit-border-image: url(border.png) 50 round; /* Safari 3.1-5 */
```

```
-o-border-image: url(border.png) 50 round; /* Opera 11-12.1 */
```

```
border-image: url(border.png) 50 round;
```

```
}
```

```
#borderimg2 {
```

```
border: 10px solid transparent;
```

```
padding: 15px;
```

```
-webkit-border-image: url(border.png) 20% round; /* Safari 3.1-5 */
```

```
-o-border-image: url(border.png) 20% round; /* Opera 11-12.1 */
```

```
border-image: url(border.png) 20% round;
}
```

```
#borderimg3 {
border: 10px solid transparent;
padding: 15px;
-webkit-border-image: url(border.png) 30% round; /* Safari 3.1-5 */
-o-border-image: url(border.png) 30% round; /* Opera 11-12.1 */
border-image: url(border.png) 30% round;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p id="borderimg1">border-image: url(border.png) 50 round;</p>
```

```
<p id="borderimg2">border-image: url(border.png) 20% round;</p>
```

```
<p id="borderimg3">border-image: url(border.png) 30% round;</p>
```

```
<p><strong>Note:</strong> Internet Explorer 10, and earlier versions, do not
support the border-image property.</p>
```

```
</body>
```


</html>

CSS3 Border Properties

Property	Description
border-image	A shorthand property for setting all the border-image-* properties
border-image-source	Specifies the path to the image to be used as a border
border-image-slice	Specifies how to slice the border image
border-image-width	Specifies the widths of the border image
border-image-outset	Specifies the amount by which the border image area extends beyond the border box
border-image-repeat	Specifies whether the border image should be repeated, rounded or stretched

CSS3 Backgrounds

CSS3 contains a few new background properties, which allow greater control of the background element.

In this chapter you will learn how to add multiple background images to one element.

You will also learn about the following new CSS3 properties:

- background-size
- background-origin
- background-clip

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit-, -moz-, or -o- specify the first version that worked with a prefix.

Property					
background-image (with multiple backgrounds)	4.0	9.0	3.6	3.1	11.5
background-size	4.0 1.0 -webkit-	9.0	4.0 3.6 -moz-	4.1 3.0 -webkit-	10.5 10.0 -o-
background-origin	1.0	9.0	4.0	3.0	10.5
background-clip	4.0	9.0	4.0	3.0	10.5

CSS3 Multiple Backgrounds

CSS3 allows you to add multiple background images for an element, through the `background-image` property.

The different background images are separated by commas, and the images are stacked on top of each other, where the first image is closest to the viewer.

The following example has two background images, the first image is a flower (aligned to the bottom and right) and the second image is a paper background (aligned to the top-left corner):

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#example1 {

    background-image: url(img_flwr.gif), url(paper.gif);

    background-position: right bottom, left top;

    background-repeat: no-repeat, repeat;

    padding: 15px;

}

</style>

</head>

<body>
```

```
<div id="example1">

<h1>Lorem Ipsum Dolor</h1>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy
nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit
lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

</body>

</html>
```

Multiple background images can be specified using either the individual background properties (as above) or the **background** shorthand property.

The following example uses the **background** shorthand property (same result as example above):

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#example1 {

    background: url(img_flwr.gif) right bottom no-repeat, url(paper.gif) left top
repeat;

    padding: 15px;
```

```
}  
</style>  
</head>  
<body>  
  
<div id="example1">  
  
<h1>Lorem Ipsum Dolor</h1>  
  
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy  
nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>  
  
<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit  
lobortis nisl ut aliquip ex ea commodo consequat.</p>  
  
</div>  
  
</body>  
</html>
```

CSS3 Background Size

The CSS3 **background-size** property allows you to specify the size of background images.

Before CSS3, the size of a background image was the actual size of the image. CSS3 allows us to re-use background images in different contexts.

The size can be specified in lengths, percentages, or by using one of the two keywords: contain or cover.

The following example resizes a background image to much smaller than the original image (using pixels):

Original background image:

Lorem Ipsum Dolor

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Ut wisi enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

Resized background image:

Lorem Ipsum Dolor

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

Ut wisi enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

Here is the code:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#example1 {
```

```
border: 1px solid black;

background:url(img_flwr.gif);

background-repeat: no-repeat;

padding:15px;

}
```

```
#example2 {

border: 1px solid black;

background:url(img_flwr.gif);

background-size: 100px 80px;

background-repeat: no-repeat;

padding:15px;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>Original background-image:</p>
```

```
<div id="example1">
```

```
<h2>Lorem Ipsum Dolor</h2>
```

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>
```

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

<p>Resized background-image:</p>

<div id="example2">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

</body>

</html>

The two other possible values for **background-size** are **contain** and **cover**.

The **contain** keyword scales the background image to be as large as possible (but both its width and its height must fit inside the content area). As such, depending on the proportions of the background image and the background positioning area, there may be some areas of the background which are not covered by the background image.

The **cover** keyword scales the background image so that the content area is completely covered by the background image (both its width and height are equal to or exceed the content

area). As such, some parts of the background image may not be visible in the background positioning area.

The following example illustrates the use of `contain` and `cover`:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.div1 {
```

```
    border: 1px solid black;
```

```
    height:150px;
```

```
    width:180px;
```

```
    background:url(img_flwr.gif);
```

```
    background-repeat: no-repeat;
```

```
}
```

```
.div2 {
```

```
    border: 1px solid black;
```

```
    height:150px;
```

```
    width:180px;
```

```
    background:url(img_flwr.gif);
```

```
    background-repeat: no-repeat;
```

```
background-size: contain;  
}
```

```
.div3 {  
    border: 1px solid black;  
    height:150px;  
    width:180px;  
    background:url(img_flwr.gif);  
    background-repeat: no-repeat;  
    background-size: cover;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>Original image:</p>
```

```
<div class="div1">
```

```
<p>Lorem ipsum dolor sit amet.</p>
```

```
</div>
```

```
<p>Using the "contain" keyword:</p>
```

```
<div class="div2">
```

```
<p>Lorem ipsum dolor sit amet.</p>
```

```
</div>
```

```
<p>Using the "cover" keyword:</p>
```

```
<div class="div3">
```

```
<p>Lorem ipsum dolor sit amet.</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

Define Sizes of Multiple Background Images

The **background-size** property also accepts multiple values for background size (using a comma-separated list), when working with multiple backgrounds.

The following example has three background images specified, with different background-size value for each image:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#example1 {
```

```
background: url(img_flwr.gif) left top no-repeat, url(img_flwr.gif) right bottom  
no-repeat, url(paper.gif) left top repeat;
```

```
padding: 15px;
```

```
background-size: 50px, 130px, auto;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div id="example1">
```

```
<h1>Lorem Ipsum Dolor</h1>
```

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy  
nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>
```

```
<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit  
lobortis nisl ut aliquip ex ea commodo consequat.</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

Full Size Background Image

Now we want to have a background image on a website that covers the entire browser window at all times.

The requirements are as follows:

- Fill the entire page with the image (no white space)
- Scale image as needed
- Center image on page
- Do not cause scrollbars

The following example shows how to do it; Use the html element (the html element is always at least the height of the browser window). Then set a fixed and centered background on it. Then adjust its size with the background-size property:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

html {

    background: url(img_flower.jpg) no-repeat center fixed;

    background-size: cover;

}

body {

    color: white;

}

</style>

</head>

<body>
```

```
<h1>Full Page Background Image</h1>
```

```
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>
```

```
</body>
```

```
</html>
```

CSS3 background-origin Property

The CSS3 `background-origin` property specifies where the background image is positioned.

The property takes three different values:

- border-box - the background image starts from the upper left corner of the border
- padding-box - (default) the background image starts from the upper left corner of the padding edge
- content-box - the background image starts from the upper left corner of the content

The following example illustrates the `background-origin` property:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#example1 {  
    border: 10px solid black;  
    padding: 35px;  
    background: url(img_flwr.gif);  
    background-repeat: no-repeat;  
}
```

```
#example2 {  
    border: 10px solid black;  
    padding: 35px;  
    background: url(img_flwr.gif);  
    background-repeat: no-repeat;  
    background-origin: border-box;  
}
```

```
#example3 {  
    border: 10px solid black;  
    padding: 35px;  
    background: url(img_flwr.gif);  
    background-repeat: no-repeat;  
    background-origin: content-box;  
}
```

</style>

</head>

<body>

<p>No background-origin (padding-box is default):</p>

<div id="example1">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

<p>background-origin: border-box:</p>

<div id="example2">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

<p>background-origin: content-box:</p>


```
<div id="example3">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p>Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</div>

</body>

</html>
```

CSS3 background-clip Property

The CSS3 **background-clip** property specifies the painting area of the background.

The property takes three different values:

- border-box - (default) the background is painted to the outside edge of the border
- padding-box - the background is painted to the outside edge of the padding
- content-box - the background is painted within the content box

The following example illustrates the **background-clip** property:

Example

```
<!DOCTYPE html>

<html>

<head>
```

```
<style>
```

```
#example1 {
```

```
    border: 10px dotted black;
```

```
    padding:35px;
```

```
    background: yellow;
```

```
}
```

```
#example2 {
```

```
    border: 10px dotted black;
```

```
    padding:35px;
```

```
    background: yellow;
```

```
    background-clip: padding-box;
```

```
}
```

```
#example3 {
```

```
    border: 10px dotted black;
```

```
    padding:35px;
```

```
    background: yellow;
```

```
    background-clip: content-box;
```

```
}
```

```
</style>
```

```
</head>
```

<body>

<p>No background-clip (border-box is default):</p>

<div id="example1">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

</div>

<p>background-clip: padding-box:</p>

<div id="example2">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

</div>

<p>background-clip: content-box:</p>

<div id="example3">

<h2>Lorem Ipsum Dolor</h2>

<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

</div>

</body>

</html>

CSS3 Background Properties

Property	Description
background	A shorthand property for setting all the background properties in one declaration
background-clip	Specifies the painting area of the background
background-image	Specifies one or more background images for an element
background-origin	Specifies where the background image(s) is/are positioned
background-size	Specifies the size of the background image(s)

CSS3 Colors

CSS supports color names, hexadecimal and RGB colors.

In addition, CSS3 also introduces:

- RGBA colors
- HSL colors
- HSLA colors
- opacity

Browser Support

The numbers in the table specify the first browser version that fully supports CSS3 color values/property.

Property					
RGBA, HSL, and HSLA	4.0	9.0	3.0	3.1	10.1
opacity	4.0	9.0	2.0	3.1	10.1

RGBA Colors

RGBA color values are an extension of RGB color values with an alpha channel - which specifies the opacity for a color.

An RGBA color value is specified with: `rgba(red, green, blue, alpha)`. The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (fully opaque).

```
rgba(255, 0, 0, 0.2);
```

```
rgba(255, 0, 0, 0.4);
```

```
rgba(255, 0, 0, 0.6);
```

```
rgba(255, 0, 0, 0.8);
```

The following example defines different RGBA colors:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#p1 {background-color:rgba(255,0,0,0.3);}

#p2 {background-color:rgba(0,255,0,0.3);}

#p3 {background-color:rgba(0,0,255,0.3);}

#p4 {background-color:rgba(192,192,192,0.3);}

#p5 {background-color:rgba(255,255,0,0.3);}

#p6 {background-color:rgba(255,0,255,0.3);}

</style>

</head>

<body>

<p>RGBA colors:</p>

<p id="p1">Red</p>

<p id="p2">Green</p>

<p id="p3">Blue</p>

<p id="p4">Grey</p>

<p id="p5">Yellow</p>
```

```
<p id="p6">Cerise</p>
```

```
</body>
```

```
</html>
```

HSL Colors

HSL stands for Hue, Saturation and Lightness.

An HSL color value is specified with: `hsl(hue, saturation, lightness)`.

1. Hue is a degree on the color wheel (from 0 to 360):
 - 0 (or 360) is red
 - 120 is green
 - 240 is blue
2. Saturation is a percentage value: 100% is the full color.
3. Lightness is also a percentage; 0% is dark (black) and 100% is white.

```
hsl(0, 100%, 30%);
```

```
hsl(0, 100%, 50%);
```

```
hsl(0, 100%, 70%);
```

```
hsl(0, 100%, 90%);
```

The following example defines different HSL colors:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#p1 {background-color:hsl(120,100%,50%);}
#p2 {background-color:hsl(120,100%,75%);}
#p3 {background-color:hsl(120,100%,25%);}
#p4 {background-color:hsl(120,60%,70%);}
#p5 {background-color:hsl(290,100%,50%);}
#p6 {background-color:hsl(290,60%,70%);}

</style>

</head>

<body>

<p>HSL colors:</p>

<p id="p1">Green</p>

<p id="p2">Light green</p>

<p id="p3">Dark green</p>

<p id="p4">Pastel green</p>

<p id="p5">Violet</p>

<p id="p6">Pastel violet</p>

</body>

</html>
```

HSLA Colors

HSLA color values are an extension of HSL color values with an alpha channel - which specifies the opacity for a color.

An HSLA color value is specified with: `hsla(hue, saturation, lightness, alpha)`, where the alpha parameter defines the opacity. The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (fully opaque).

```
hsla(0, 100%, 30%, 0.3);
```

```
hsla(0, 100%, 50%, 0.3);
```

```
hsla(0, 100%, 70%, 0.3);
```

```
hsla(0, 100%, 90%, 0.3);
```

The following example defines different HSLA colors:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#p1 {background-color:hsla(120,100%,50%,0.3);}

#p2 {background-color:hsla(120,100%,75%,0.3);}

#p3 {background-color:hsla(120,100%,25%,0.3);}

#p4 {background-color:hsla(120,60%,70%,0.3);}

#p5 {background-color:hsla(290,100%,50%,0.3);}

#p6 {background-color:hsla(290,60%,70%,0.3);}

</style>

</head>
```

```
<body>
```

```
<p>HSLA colors:</p>
```

```
<p id="p1">Green</p>
```

```
<p id="p2">Light green</p>
```

```
<p id="p3">Dark green</p>
```

```
<p id="p4">Pastel green</p>
```

```
<p id="p5">Violet</p>
```

```
<p id="p6">Pastel violet</p>
```

```
</body>
```

```
</html>
```

Opacity

The CSS3 `opacity` property sets the opacity for a specified RGB value.

The `opacity` property value must be a number between 0.0 (fully transparent) and 1.0 (fully opaque).

```
rgb(255, 0, 0);opacity:0.2;  
rgb(255, 0, 0);opacity:0.4;  
rgb(255, 0, 0);opacity:0.6;  
rgb(255, 0, 0);opacity:0.8;
```

Notice that the text above will also be opaque.

The following example shows different RGB values with opacity:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#p1 {background-color:rgb(255,0,0);opacity:0.6;}

#p2 {background-color:rgb(0,255,0);opacity:0.6;}

#p3 {background-color:rgb(0,0,255);opacity:0.6;}

#p4 {background-color:rgb(192,192,192);opacity:0.6;}

#p5 {background-color:rgb(255,255,0);opacity:0.6;}

#p6 {background-color:rgb(255,0,255);opacity:0.6;}

</style>

</head>

<body>


<p>RGB colors with opacity:</p>

<p id="p1">Red</p>

<p id="p2">Green</p>

<p id="p3">Blue</p>

<p id="p4">Grey</p>

<p id="p5">Yellow</p>

<p id="p6">Cerise</p>
```

</body>

</html>

Gradient Background

CSS3 gradients let you display smooth transitions between two or more specified colors.

Earlier, you had to use images for these effects. However, by using CSS3 gradients you can reduce download time and bandwidth usage. In addition, elements with gradients look better when zoomed, because the gradient is generated by the browser.

CSS3 defines two types of gradients:

- **Linear Gradients (goes down/up/left/right/diagonally)**
- **Radial Gradients (defined by their center)**

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit-, -moz-, or -o- specify the first version that worked with a prefix.

Property			
linear-gradient	26.0 10.0 -webkit-	10.0	16.0 3.6 -moz-

radial-gradient	26.0 10.0 -webkit-	10.0	16.0 3.6 -moz-
repeating-linear-gradient	26.0 10.0 -webkit-	10.0	16.0 3.6 -moz-
repeating-radial-gradient	26.0 10.0 -webkit-	10.0	16.0 3.6 -moz-

CSS3 Linear Gradients

To create a linear gradient you must define at least two color stops. Color stops are the colors you want to render smooth transitions among. You can also set a starting point and a direction (or an angle) along with the gradient effect.

Syntax

background: linear-gradient(*direction, color-stop1, color-stop2, ...*);

Linear Gradient - Top to Bottom (this is default)

The following example shows a linear gradient that starts at the top. It starts red, transitioning to yellow:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
height: 200px;

background: red; /* For browsers that do not support gradients */

background: -webkit-linear-gradient(red, yellow); /* For Safari 5.1 to 6.0 */

background: -o-linear-gradient(red, yellow); /* For Opera 11.1 to 12.0 */

background: -moz-linear-gradient(red, yellow); /* For Firefox 3.6 to 15 */

background: linear-gradient(red, yellow); /* Standard syntax (must be last) */
}

</style>

</head>

<body>

<h3>Linear Gradient - Top to Bottom</h3>

<p>This linear gradient starts at the top. It starts red, transitioning to yellow:</p>

<div id="grad1"></div>

<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not
support gradients.</p>

</body>

</html>
```

Linear Gradient - Left to Right

The following example shows a linear gradient that starts from the left. It starts red, transitioning to yellow:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#grad1 {

    height: 200px;

    background: red; /* For browsers that do not support gradients */

    background: -webkit-linear-gradient(left, red , yellow); /* For Safari 5.1 to 6.0 */

    background: -o-linear-gradient(right, red, yellow); /* For Opera 11.1 to 12.0 */

    background: -moz-linear-gradient(right, red, yellow); /* For Firefox 3.6 to 15 */

    background: linear-gradient(to right, red , yellow); /* Standard syntax (must be

last) */

}

</style>

</head>

<body>

<h3>Linear Gradient - Left to Right</h3>

<p>This linear gradient starts at the left. It starts red, transitioning to yellow:</p>
```

```
<div id="grad1"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

Linear Gradient - Diagonal

You can make a gradient diagonally by specifying both the horizontal and vertical starting positions.

The following example shows a linear gradient that starts at top left (and goes to bottom right). It starts red, transitioning to yellow:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 200px;
```

```
    background: red; /* For browsers that do not support gradients */
```

```
    background: -webkit-linear-gradient(left top, red, yellow); /* For Safari 5.1 to  
6.0 */
```



```
background: -o-linear-gradient(bottom right, red, yellow); /* For Opera 11.1 to 12.0 */
```

```
background: -moz-linear-gradient(bottom right, red, yellow); /* For Firefox 3.6 to 15 */
```

```
background: linear-gradient(to bottom right, red, yellow); /* Standard syntax (must be last) */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Linear Gradient - Diagonal</h3>
```

```
<p>This linear gradient starts at top left. It starts red, transitioning to yellow:</p>
```

```
<div id="grad1"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not support gradients.</p>
```

```
</body>
```

```
</html>
```

Using Angles

If you want more control over the direction of the gradient, you can define an angle, instead of the predefined directions (to bottom, to top, to right, to left, to bottom right, etc.).

Syntax

background: linear-gradient(*angle*, *color-stop1*, *color-stop2*);

The angle is specified as an angle between a horizontal line and the gradient line.

The following example shows how to use angles on linear gradients:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#grad1 {

    height: 100px;

    background: red; /* For browsers that do not support gradients */

    background: -webkit-linear-gradient(0deg, red, yellow); /* For Safari 5.1 to 6.0 */

    background: -o-linear-gradient(0deg, red, yellow); /* For Opera 11.1 to 12.0 */

    background: -moz-linear-gradient(0deg, red, yellow); /* For Firefox 3.6 to 15 */

    background: linear-gradient(0deg, red, yellow); /* Standard syntax (must be last) */

}
```

```
#grad2 {  
    height: 100px;  
    background: red; /* For browsers that do not support gradients */  
    background: -webkit-linear-gradient(90deg, red, yellow); /* For Safari 5.1 to 6.0  
*/  
    background: -o-linear-gradient(90deg, red, yellow); /* For Opera 11.1 to 12.0 */  
    background: -moz-linear-gradient(90deg, red, yellow); /* For Firefox 3.6 to 15  
*/  
    background: linear-gradient(90deg, red, yellow); /* Standard syntax (must be  
last) */  
}
```

```
#grad3 {  
    height: 100px;  
    background: red; /* For browsers that do not support gradients */  
    background: -webkit-linear-gradient(180deg, red, yellow); /* For Safari 5.1 to  
6.0 */  
    background: -o-linear-gradient(180deg, red, yellow); /* For Opera 11.1 to 12.0  
*/  
    background: -moz-linear-gradient(180deg, red, yellow); /* For Firefox 3.6 to 15  
*/  
    background: linear-gradient(180deg, red, yellow); /* Standard syntax (must be  
last) */
```

```
}
```

```
#grad4 {
```

```
    height: 100px;
```

```
    background: red; /* For browsers that do not support gradients */
```

```
    background: -webkit-linear-gradient(-90deg, red, yellow); /* For Safari 5.1 to 6.0 */
```

```
    background: -o-linear-gradient(-90deg, red, yellow); /* For Opera 11.1 to 12.0 */
```

```
    background: -moz-linear-gradient(-90deg, red, yellow); /* For Firefox 3.6 to 15 */
```

```
    background: linear-gradient(-90deg, red, yellow); /* Standard syntax (must be last) */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Linear Gradients - Using Different Angles</h3>
```

```
<div id="grad1" style="color:white;text-align:center;">0deg</div><br>
```

```
<div id="grad2" style="color:white;text-align:center;">90deg</div><br>
```

```
<div id="grad3" style="color:white;text-align:center;">180deg</div><br>
```

```
<div id="grad4" style="color:white;text-align:center;">-90deg</div>
```

Note: Internet Explorer 9 and earlier versions do not support gradients.

</body>

</html>

Using Multiple Color Stops

The following example shows a linear gradient (from top to bottom) with multiple color stops:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 200px;
```

```
    background: -webkit-linear-gradient(red, yellow, green); /* For Safari 5.1 to 6.0 */
```

```
    background: -o-linear-gradient(red, yellow, green); /* For Opera 11.1 to 12.0 */
```

```
    background: -moz-linear-gradient(red, yellow, green); /* For Firefox 3.6 to 15 */
```

```
    background: linear-gradient(red, yellow, green); /* Standard syntax (must be last) */
```

```
}
```

```
#grad2 {  
  
    height: 200px;  
  
    background: -webkit-linear-gradient(red, orange, yellow, green, blue, indigo,  
violet); /* For Safari 5.1 to 6.0 */  
  
    background: -o-linear-gradient(red, orange, yellow, green, blue, indigo, violet);  
/* For Opera 11.1 to 12.0 */  
  
    background: -moz-linear-gradient(red, orange, yellow, green, blue, indigo,  
violet); /* For Firefox 3.6 to 15 */  
  
    background: linear-gradient(red, orange, yellow, green, blue, indigo, violet); /*  
Standard syntax (must be last) */  
}
```

```
#grad3 {  
  
    height: 200px;  
  
    background: -webkit-linear-gradient(red 10%, green 85%, blue 90%); /* For  
Safari 5.1 to 6.0 */  
  
    background: -o-linear-gradient(red 10%, green 85%, blue 90%); /* For Opera  
11.1 to 12.0 */  
  
    background: -moz-linear-gradient(red 10%, green 85%, blue 90%); /* For  
Firefox 3.6 to 15 */  
  
    background: linear-gradient(red 10%, green 85%, blue 90%); /* Standard syntax  
(must be last) */  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>3 Color Stops (evenly spaced)</h3>
```

```
<div id="grad1"></div>
```

```
<h3>7 Color Stops (evenly spaced)</h3>
```

```
<div id="grad2"></div>
```

```
<h3>3 Color Stops (not evenly spaced)</h3>
```

```
<div id="grad3"></div>
```

```
<p><strong>Note:</strong> Color stops are automatically spaced evenly when no  
percents are specified.</p>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

The following example shows how to create a linear gradient (from left to right) with the color of the rainbow and some text:

Gradient Background

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#grad1 {

    height: 55px;

    background: -webkit-linear-gradient(left, red, orange, yellow, green, blue,
indigo, violet); /* For Safari 5.1 to 6.0 */

    background: -o-linear-gradient(left, red, orange, yellow, green, blue, indigo,
violet); /* For Opera 11.1 to 12.0 */

    background: -moz-linear-gradient(left, red, orange, yellow, green, blue, indigo,
violet); /* For Fx 3.6 to 15 */

    background: linear-gradient(to right, red, orange, yellow, green, blue, indigo,
violet); /* Standard syntax (must be last) */

}

</style>

</head>

<body>

<div id="grad1" style="text-align:center;margin:auto;color:#888888;font-
size:40px;font-weight:bold">

Gradient Background

</div>
```


Note: Internet Explorer 9 and earlier versions do not support gradients.

</body>

</html>

Using Transparency

CSS3 gradients also support transparency, which can be used to create fading effects.

To add transparency, we use the `rgba()` function to define the color stops. The last parameter in the `rgba()` function can be a value from 0 to 1, and it defines the transparency of the color: 0 indicates full transparency, 1 indicates full color (no transparency).

The following example shows a linear gradient that starts from the left. It starts fully transparent, transitioning to full color red:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 200px;
```

```
    background: -webkit-linear-gradient(left, rgba(255,0,0,0), rgba(255,0,0,1)); /*
```

```
For Safari 5.1 to 6.0 */
```

```
background: -o-linear-gradient(right, rgba(255,0,0,0), rgba(255,0,0,1)); /* For  
Opera 11.1 to 12.0 */
```

```
background: -moz-linear-gradient(right, rgba(255,0,0,0), rgba(255,0,0,1)); /* For  
Firefox 3.6 to 15 */
```

```
background: linear-gradient(to right, rgba(255,0,0,0), rgba(255,0,0,1)); /*  
Standard syntax (must be last) */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Linear Gradient - Transparency</h3>
```

```
<p>To add transparency, we use the rgba() function to define the color stops. The  
last parameter in the rgba() function can be a value from 0 to 1, and it defines the  
transparency of the color: 0 indicates full transparency, 1 indicates full color (no  
transparency).</p>
```

```
<div id="grad1"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

Repeating a linear-gradient

The `repeating-linear-gradient()` function is used to repeat linear gradients:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#grad1 {

    height: 200px;

    background: -webkit-repeating-linear-gradient(red, yellow 10%, green 20%); /*
For Safari 5.1 to 6.0 */

    background: -o-repeating-linear-gradient(red, yellow 10%, green 20%); /* For
Opera 11.1 to 12.0 */

    background: -moz-repeating-linear-gradient(red, yellow 10%, green 20%); /*
For Firefox 3.6 to 15 */

    background: repeating-linear-gradient(red, yellow 10%, green 20%); /*
Standard syntax (must be last) */

}

#grad2 {

    height: 200px;

    background: -webkit-repeating-linear-gradient(45deg,red,yellow 7%,green
10%); /* For Safari 5.1 to 6.0 */
```

```
background: -o-repeating-linear-gradient(45deg,red,yellow 7%,green 10%); /*  
For Opera 11.1 to 12.0 */
```

```
background: -moz-repeating-linear-gradient(45deg,red,yellow 7%,green 10%);  
/* For Firefox 3.6 to 15 */
```

```
background: repeating-linear-gradient(45deg,red,yellow 7%,green 10%); /*  
Standard syntax (must be last) */
```

```
}
```

```
#grad3 {
```

```
height: 200px;
```

```
background: -webkit-repeating-linear-gradient(190deg,red,yellow 7%,green  
10%); /* For Safari 5.1 to 6.0 */
```

```
background: -o-repeating-linear-gradient(190deg,red,yellow 7%,green 10%); /*  
For Opera 11.1 to 12.0 */
```

```
background: -moz-repeating-linear-gradient(190deg,red,yellow 7%,green 10%);  
/* For Firefox 3.6 to 15 */
```

```
background: repeating-linear-gradient(190deg,red,yellow 7%,green 10%); /*  
Standard syntax (must be last) */
```

```
}
```

```
#grad4 {
```

```
height: 200px;
```

```
background: -webkit-repeating-linear-gradient(90deg,red,yellow 7%,green  
10%); /* For Safari 5.1 to 6.0 */
```

```
background: -o-repeating-linear-gradient(); /* For Opera 11.1 to 12.0 */

background: -moz-repeating-linear-gradient(90deg,red,yellow 7%,green 10%);
/* For Firefox 3.6 to 15 */

background: repeating-linear-gradient(90deg,red,yellow 7%,green 10%); /*
Standard syntax (must be last) */
}

</style>

</head>

<body>
```

```
<h3>Repeating Linear Gradient</h3>
```

```
<div id="grad1"></div>
```

```
<p>A repeating gradient on 45deg axe starting red and finishing green:</p>
```

```
<div id="grad2"></div>
```

```
<p>A repeating gradient on 190deg axe starting red and finishing green:</p>
```

```
<div id="grad3"></div>
```

```
<p>A repeating gradient on 90deg axe starting red and finishing green:</p>
```

```
<div id="grad4"></div>
```

<p>Note: Internet Explorer 9 and earlier versions do not support gradients.</p>

</body>

</html>

CSS3 Radial Gradients

A radial gradient is defined by its center.

To create a radial gradient you must also define at least two color stops.

Syntax

background: radial-gradient(*shape size at position, start-color, ..., last-color*);

By default, shape is ellipse, size is farthest-corner, and position is center.

Radial Gradient - Evenly Spaced Color Stops (this is default)

The following example shows a radial gradient with evenly spaced color stops:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {  
    height: 150px;  
    width: 200px;  
    background: red; /* For browsers that do not support gradients */  
    background: -webkit-radial-gradient(red, yellow, green); /* Safari 5.1 to 6.0 */  
    background: -o-radial-gradient(red, yellow, green); /* For Opera 11.6 to 12.0 */  
    background: -moz-radial-gradient(red, yellow, green); /* For Firefox 3.6 to 15 */  
    background: radial-gradient(red, yellow, green); /* Standard syntax (must be  
last) */  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Radial Gradient - Evenly Spaced Color Stops</h3>
```

```
<div id="grad1"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

Radial Gradient - Differently Spaced Color Stops

The following example shows a radial gradient with differently spaced color stops:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

#grad1 {

    height: 150px;

    width: 200px;

    background: red; /* For browsers that do not support gradients */

    background: -webkit-radial-gradient(red 5%, yellow 15%, green 60%); /* Safari 5.1-6.0 */

    background: -o-radial-gradient(red 5%, yellow 15%, green 60%); /* For Opera 11.6 to 12.0 */

    background: -moz-radial-gradient(red 5%, yellow 15%, green 60%); /* For Firefox 3.6 to 15 */

    background: radial-gradient(red 5%, yellow 15%, green 60%); /* Standard syntax (must be last) */

}

</style>

</head>

<body>
```


<h3>Radial Gradient - Differently Spaced Color Stops</h3>

<div id="grad1"></div>

<p>Note: Internet Explorer 9 and earlier versions do not support gradients.</p>

</body>

</html>

Set Shape

The shape parameter defines the shape. It can take the value circle or ellipse. The default value is ellipse.

The following example shows a radial gradient with the shape of a circle:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 150px;
```

```
    width: 200px;
```

```
background: -webkit-radial-gradient(red, yellow, green); /* For Safari 5.1 to 6.0
*/
background: -o-radial-gradient(red, yellow, green); /* For Opera 11.6 to 12.0 */
background: -moz-radial-gradient(red, yellow, green); /* For Fx 3.6 to 15 */
background: radial-gradient(red, yellow, green); /* Standard syntax (must be
last) */
}
```

```
#grad2 {
    height: 150px;
    width: 200px;

    background: -webkit-radial-gradient(circle, red, yellow, green); /* For Safari 5.1
to 6.0 */
    background: -o-radial-gradient(circle, red, yellow, green); /* For Opera 11.6 to
12.0 */
    background: -moz-radial-gradient(circle, red, yellow, green); /* For Fx 3.6 to 15
*/
    background: radial-gradient(circle, red, yellow, green); /* Standard syntax
(must be last) */
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Radial Gradient - Shapes</h3>
```

```
<p><strong>Ellipse (this is default):</strong></p>
```

```
<div id="grad1"></div>
```

```
<p><strong>Circle:</strong></p>
```

```
<div id="grad2"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

Use of Different Size Keywords

The size parameter defines the size of the gradient. It can take four values:

- **closest-side**
- **farthest-side**
- **closest-corner**
- **farthest-corner**

Example

A radial gradient with different size keywords:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 150px;
```

```
    width: 150px;
```

```
    background: -webkit-radial-gradient(60% 55%, closest-side, red, yellow, black);  
    /* Safari 5.1 to 6.0 */
```

```
    background: -o-radial-gradient(60% 55%, closest-side, red, yellow, black); /* For  
    Opera 11.6 to 12.0 */
```

```
    background: -moz-radial-gradient(60% 55%, closest-side, red, yellow, black); /*  
    For Firefox 3.6 to 15 */
```

```
    background: radial-gradient(closest-side at 60% 55%, red, yellow, black); /*  
    Standard syntax (must be last) */
```

```
}
```

```
#grad2 {
```

```
    height: 150px;
```

```
    width: 150px;
```

```
    background: -webkit-radial-gradient(60% 55%, farthest-side, red, yellow, black);  
    /* Safari 5.1 to 6.0 */
```

```
    background: -o-radial-gradient(60% 55%, farthest-side, red, yellow, black); /*  
    For Opera 11.6 to 12.0 */
```

```
    background: -moz-radial-gradient(60% 55%, farthest-side, red, yellow, black); /*  
    For Firefox 3.6 to 15 */
```

```
    background: radial-gradient(farthest-side at 60% 55%, red, yellow, black); /*
Standard syntax (must be last) */
}
```

```
#grad3 {

    height: 150px;

    width: 150px;

    background: -webkit-radial-gradient(60% 55%, closest-corner, red, yellow,
black); /* Safari 5.1 to 6.0 */

    background: -o-radial-gradient(60% 55%, closest-corner, red, yellow, black); /*
For Opera 11.6 to 12.0 */

    background: -moz-radial-gradient(60% 55%, closest-corner, red, yellow, black);
/* For Firefox 3.6 to 15 */

    background: radial-gradient(closest-corner at 60% 55%, red, yellow, black); /*
Standard syntax (must be last) */

}
```

```
#grad4 {

    height: 150px;

    width: 150px;

    background: -webkit-radial-gradient(60% 55%, farthest-corner, red, yellow,
black); /* Safari 5.1 to 6.0 */

    background: -o-radial-gradient(60% 55%, farthest-corner, red, yellow, black); /*
For Opera 11.6 to 12.0 */

}
```

```
background: -moz-radial-gradient(60% 55%, farthest-corner, red, yellow, black);  
/* For Firefox 3.6 to 15 */
```

```
background: radial-gradient(farthest-corner at 60% 55%, red, yellow, black); /*  
Standard syntax (must be last) */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Radial Gradients - Use of different size keywords</h3>
```

```
<p><strong>closest-side:</strong></p>
```

```
<div id="grad1"></div>
```

```
<p><strong>farthest-side:</strong></p>
```

```
<div id="grad2"></div>
```

```
<p><strong>closest-corner:</strong></p>
```

```
<div id="grad3"></div>
```

```
<p><strong>farthest-corner (this is default):</strong></p>
```

```
<div id="grad4"></div>
```

<p>Note: Internet Explorer 9 and earlier versions do not support gradients.</p>

</body>

</html>

Repeating a radial-gradient

The repeating-radial-gradient() function is used to repeat radial gradients:

Example

A repeating radial gradient:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#grad1 {
```

```
    height: 150px;
```

```
    width: 200px;
```

```
    background: -webkit-repeating-radial-gradient(red, yellow 10%, green 15%); /*  
For Safari 5.1 to 6.0 */
```

```
    background: -o-repeating-radial-gradient(red, yellow 10%, green 15%); /* For  
Opera 11.6 to 12.0 */
```

```
background: -moz-repeating-radial-gradient(red, yellow 10%, green 15%); /*  
For Firefox 3.6 to 15 */
```

```
background: repeating-radial-gradient(red, yellow 10%, green 15%); /*  
Standard syntax (must be last) */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3>Repeating Radial Gradient</h3>
```

```
<div id="grad1"></div>
```

```
<p><strong>Note:</strong> Internet Explorer 9 and earlier versions do not  
support gradients.</p>
```

```
</body>
```

```
</html>
```

CSS3 Shadow Effects

With CSS3 you can add shadow to text and to elements.

In this chapter you will learn about the following properties:

- `text-shadow`
- `box-shadow`

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by `-webkit-` or `-moz-` specifies the first version that worked with a prefix.

Property					
text-shadow	4.0	10.0	3.5	4.0	9.5
box-shadow	10.0 4.0 -webkit-	9.0	4.0 3.5 -moz-	5.1 3.1 -webkit-	10.5

CSS3 Text Shadow

The CSS3 `text-shadow` property applies shadow to text.

In its simplest use, you only specify the horizontal shadow (2px) and the vertical shadow (2px):

Text shadow effect!

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
    text-shadow: 2px 2px;
}
</style>
</head>
<body>

<h1>Text-shadow effect!</h1>
```

<p>Note: Internet Explorer 9 and earlier versions, do not support the text-shadow property.</p>

```
</body>
```

```
</html>
```

Text shadow effect!

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

h1 {

    text-shadow: 2px 2px red;

}

</style>
```

```
</head>
```

```
<body>
```

```
<h1>Text-shadow effect!</h1>
```

```
<p><b>Note:</b> Internet Explorer 9 and earlier versions, do not support the  
text-shadow property.</p>
```

```
</body>
```

```
</html>
```

Text shadow effect!

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
    text-shadow: 2px 2px 5px red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Text-shadow effect!</h1>
```

```
<p><b>Note:</b> Internet Explorer 9 and earlier versions, do not support the  
text-shadow property.</p>
```

```
</body>
```

```
</html>
```

The following example shows a white text with black shadow:

Example

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<style>  
h1 {  
    color: white;  
    text-shadow: 2px 2px 4px #000000;  
}  
</style>  
</head>  
<body>
```

```
<h1>Text-shadow effect!</h1>
```

```
<p><b>Note:</b> Internet Explorer 9 and earlier versions, do  
not support the text-shadow property.</p>
```

```
</body>
```

```
</html>
```

The following example shows a red neon glow shadow:

Text shadow effect!

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
    text-shadow: 0 0 3px #FF0000;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Text-shadow effect!</h1>
```

Note: Internet Explorer 9 and earlier versions, do not support the text-shadow property.

</body>

</html>

Multiple Shadows

To add more than one shadow to the text, you can add a comma-separated list of shadows.

The following example shows a red and blue neon glow shadow:

Text shadow effect!

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
    text-shadow: 0 0 3px #FF0000, 0 0 5px #0000FF;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Text-shadow effect!</h1>
```

```
<p><b>Note:</b> Internet Explorer 9 and earlier versions, do not support the  
text-shadow property.</p>
```

```
</body>
```

```
</html>
```

The following example shows a white text with black, blue, and darkblue shadow:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
    color: white;
```

```
    text-shadow: 1px 1px 2px black, 0 0 25px blue, 0 0 5px darkblue;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Text-shadow effect!</h1>
```

```
<p><b>Note:</b> Internet Explorer 9 and earlier versions, do not support the  
text-shadow property.</p>
```

```
</body>
```

```
</html>
```

CSS3 box-shadow Property

The CSS3 `box-shadow` property applies shadow to elements.

In its simplest use, you only specify the horizontal shadow and the vertical shadow:

This is a yellow `<div>` element with a black box-shadow

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 15px;
```

```
    background-color: yellow;
```

```
    box-shadow: 10px 10px;
```



```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>This is a div element with a box-shadow</div>
```

```
</body>
```

```
</html>
```

Next, add a color to the shadow:

```
This is a yellow <div> element with a grey box-shadow
```

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 15px;
```

```
    background-color: yellow;
```

```
    box-shadow: 10px 10px grey;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>This is a div element with a box-shadow</div>
```

```
</body>
```

```
</html>
```

Next, add a blur effect to the shadow:

This is a yellow <div> element with a blurred, grey box-shadow

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 15px;
```

```
    background-color: yellow;
```

```
    box-shadow: 10px 10px 5px grey;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>This is a div element with a box-shadow</div>
```

```
</body>
```

```
</html>
```

You can also add shadows to the `::before` and `::after` pseudo-classes, to create an interesting effect:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#boxshadow {
```

```
    position: relative;
```

```
    -moz-box-shadow: 1px 2px 4px rgba(0, 0, 0,0.5);
```

```
    -webkit-box-shadow: 1px 2px 4px rgba(0, 0, 0, .5);
```

```
    box-shadow: 1px 2px 4px rgba(0, 0, 0, .5);
```

```
    padding: 10px;
```

```
background: white;

}
```

```
/* Make the image fit the box */
```

```
#boxshadow img {

    width: 100%;

    border: 1px solid #8a4419;

    border-style: inset;

}
```

```
#boxshadow::after {

    content: "";

    position: absolute;

    z-index: -1; /* hide shadow behind image */

    -webkit-box-shadow: 0 15px 20px rgba(0, 0, 0, 0.3);

    -moz-box-shadow: 0 15px 20px rgba(0, 0, 0, 0.3);

    box-shadow: 0 15px 20px rgba(0, 0, 0, 0.3);

    width: 70%;

    left: 15%; /* one half of the remaining 30% */

    height: 100px;

    bottom: 0;

}
```

```
</style>

</head>

<body>

<div id="boxshadow">

</div>

</body>

</html>
```

Cards

An example of using the `box-shadow` property to create paper-like cards:



1

January 1, 2016



Hardanger, Norway

Example

Text card

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div.card {
```

```
  width: 250px;
```

```
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.19);
```

```
  text-align: center;
```

```
}
```

```
div.header {  
    background-color: #4CAF50;  
    color: white;  
    padding: 10px;  
    font-size: 40px;  
}
```

```
div.container {  
    padding: 10px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Cards</h2>
```

```
<p>The box-shadow property can be used to create paper-like cards:</p>
```

```
<div class="card">
```

```
  <div class="header">
```

```
<h1>1</h1>
```

```
</div>
```

```
<div class="container">
```

```
<p>January 1, 2016</p>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

ImaGE CARD

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div.polaroid {
```

```
width: 250px;
```

```
box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.19);
```

```
text-align: center;
```

```
}
```

```
div.container {
```



```
padding: 10px;
}
</style>
</head>
<body>

<h2>Polaroid Images / Cards</h2>

<p>The box-shadow property can be used to create paper-like cards:</p>

<div class="polaroid">
  
  <div class="container">
    <p>Hardanger, Norway</p>
  </div>
</div>

</body>
</html>
```

CSS3 Shadow Properties

The following table lists the CSS3 shadow properties:

Property	Description
box-shadow	Adds one or more shadows to an element
text-shadow	Adds one or more shadows to a text

CSS3 Text

CSS3 contains several new text features.

In this chapter you will learn about the following text properties:

- [text-overflow](#)
- [word-wrap](#)
- [word-break](#)

CSS3 Text Overflow

The CSS3 [text-overflow](#) property specifies how overflowed content that is not displayed should be signaled to the user.

It can be clipped:

This is some long text that will not fit in the box

or it can be rendered as an ellipsis (...):

This is some long text that will not fit in the box

The CSS code is as follows:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p.test1 {  
    white-space: nowrap;  
    width: 200px;  
    border: 1px solid #000000;  
    overflow: hidden;  
    text-overflow: clip;  
}
```

```
p.test2 {  
    white-space: nowrap;  
    width: 200px;  
    border: 1px solid #000000;  
    overflow: hidden;  
    text-overflow: ellipsis;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>The following two paragraphs contains a long text that will not fit in the box.</p>

<p>text-overflow: clip:</p>

<p class="test1">This is some long text that will not fit in the box</p>

<p>text-overflow: ellipsis:</p>

<p class="test2">This is some long text that will not fit in the box</p>

</body>

</html>

The following example shows how you can display the overflowed content when hovering over the element:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div.test {
```

```
    white-space: nowrap;
```

```
    width: 200px;
```

```
    overflow: hidden;
```

```
    border: 1px solid #000000;
```

```
}
```

```
div.test:hover {  
    text-overflow: inherit;  
    overflow: visible;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>Hover over the two divs below, to see the entire text.</p>
```

```
<div class="test" style="text-overflow:ellipsis;">This is some long text that will not  
fit in the box</div>
```

```
<br>
```

```
<div class="test" style="text-overflow:clip;">This is some long text that will not fit  
in the box</div>
```

```
</body>
```

```
</html>
```

CSS3 Word Wrapping

The CSS3 **word-wrap** property allows long words to be able to be broken and wrap onto the next line.

If a word is too long to fit within an area, it expands outside:

This paragraph contains a very long word:
thisisaveryveryveryveryveryverylongword. The long word will break and wrap to the next line.

The word-wrap property allows you to force the text to wrap - even if it means splitting it in the middle of a word:

This paragraph contains a very long word:
thisisaveryveryveryveryveryverylongword. The long word will break and wrap to the next line.

The CSS code is as follows:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

p.test {

    width: 11em;

    border: 1px solid #000000;

    word-wrap: break-word;

}

</style>

</head>

<body>
```

```
<p class="test"> This paragraph contains a very long word:  
thisisaveryveryveryveryveryverylongword. The long word will break and wrap to  
the next line.</p>
```

```
</body>
```

```
</html>
```

CSS3 Word Breaking

The CSS3 **word-break** property specifies line breaking rules.

This paragraph contains some text. This line will-break-at-hyphens.

This paragraph contains some text. The lines will break at any character.

The CSS code is as follows:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p.test1 {
```

```
    width: 140px;
```

```
    border: 1px solid #000000;
```

```
    word-break: keep-all;
```

```
}
```

```
p.test2 {  
    width: 140px;  
    border: 1px solid #000000;  
    word-break: break-all;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p class="test1">This paragraph contains some text. This line will-break-at-  
hyphens.</p>
```

```
<p class="test2">This paragraph contains some text. The lines will break at any  
character.</p>
```

```
<p><b>Note:</b> The word-break property is not supported in Opera 12 and  
earlier versions.</p>
```

```
</body>
```

```
</html>
```

CSS3 Text Properties

The following table lists the new CSS3 text properties:

Property	Description
text-align-last	Specifies how to align the last line of a text
text-emphasis	A shorthand for setting text-emphasis-style and text-emphasis-color in one declaration
text-justify	Specifies how justified text should be aligned and spaced
text-overflow	Specifies how overflowed content that is not displayed should be signaled to the user
word-break	Specifies line breaking rules for non-CJK scripts
word-wrap	Allows long words to be able to be broken and wrap onto the next line

CSS3 Web Fonts - The @font-face Rule

Web fonts allow Web designers to use fonts that are not installed on the user's computer.

When you have found/bought the font you wish to use, just include the font file on your web server, and it will be automatically downloaded to the user when needed.

Your "own" fonts are defined within the CSS3 `@font-face` rule.

Different Font Formats

TrueType Fonts (TTF)

TrueType is a font standard developed in the late 1980s, by Apple and Microsoft. TrueType is the most common font format for both the Mac OS and Microsoft Windows operating systems.

OpenType Fonts (OTF)

OpenType is a format for scalable computer fonts. It was built on TrueType, and is a registered trademark of Microsoft. OpenType fonts are used commonly today on the major computer platforms.

The Web Open Font Format (WOFF)

WOFF is a font format for use in web pages. It was developed in 2009, and is now a W3C Recommendation. WOFF is essentially OpenType or TrueType with compression and additional metadata. The goal is to support font distribution from a server to a client over a network with bandwidth constraints.

The Web Open Font Format (WOFF 2.0)

TrueType/OpenType font that provides better compression than WOFF 1.0.

SVG Fonts/Shapes

SVG fonts allow SVG to be used as glyphs when displaying text. The SVG 1.1 specification define a font module that allows the creation of fonts within an SVG document. You can also apply CSS to SVG documents, and the @font-face rule can be applied to text in SVG documents.

Embedded OpenType Fonts (EOT)

EOT fonts are a compact form of OpenType fonts designed by Microsoft for use as embedded fonts on web pages.

Using The Font You Want

In the CSS3 **@font-face** rule you must first define a name for the font (e.g. myFirstFont), and then point to the font file.

Tip: Always use lowercase letters for the font URL. Uppercase letters can give unexpected results in IE.

To use the font for an HTML element, refer to the name of the font (myFirstFont) through the **font-family** property:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
@font-face {
```

```
    font-family: myFirstFont;
```

```
    src: url(sansation_light.woff);
```

```
}
```

```
div {
```

```
    font-family: myFirstFont;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

With CSS3, websites can finally use fonts other than the pre-selected "web-safe" fonts.

```
</div>
```

<p>Note: Internet Explorer 8 and earlier, do not support the @font-face rule.</p>

</body>

</html>

Using Bold Text

You must add another @font-face rule containing descriptors for bold text:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
@font-face {
```

```
    font-family: myFirstFont;
```

```
    src: url(sansation_light.woff);
```

```
}
```

```
@font-face {
```

```
    font-family: myFirstFont;
```

```
    src: url(sansation_bold.woff);
```

```
    font-weight: bold;
```

```
}
```

```
div {  
    font-family: myFirstFont;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

With CSS3, websites can **finally** use fonts other than the pre-selected "web-safe" fonts.

```
</div>
```

Note: Internet Explorer 8 and earlier, do not support the @font-face rule.

```
</body>
```

```
</html>
```

CSS3 Font Descriptors

The following table lists all the font descriptors that can be defined inside the **@font-face** rule:

Descriptor	Values	Description
font-family	<i>name</i>	Required. Defines a name for the font
src	<i>URL</i>	Required. Defines the URL of the font file
font-stretch	normal condensed ultra-condensed extra-condensed semi-condensed expanded semi-expanded extra-expanded ultra-expanded	Optional. Defines how the font should be stretched. Default is "normal"
font-style	normal italic oblique	Optional. Defines how the font should be styled. Default is "normal"
font-weight	normal bold 100 200 300 400 500 600 700 800 900	Optional. Defines the boldness of the font. Default is "normal"
unicode-range	<i>unicode-range</i>	Optional. Defines the range of UNICODE characters the font supports. Default is "U+0-10FFFF"

CSS3 2D Transforms

[< Previous](#) [Next >](#)

CSS3 Transforms

CSS3 transforms allow you to translate, rotate, scale, and skew elements.

A transformation is an effect that lets an element change shape, size and position.

CSS3 supports 2D and 3D transformations.

Mouse over the elements below to see the difference between a 2D and a 3D transformation:

2D rotate

3D rotate

Browser Support for 2D Transforms

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -ms-, -webkit-, -moz-, or -o- specify the first version that worked with a prefix.

CSS3 2D Transforms

In this chapter you will learn about the following 2D transformation methods:

- `translate()`
- `rotate()`
- `scale()`
- `skewX()`
- `skewY()`

- `matrix()`

Tip: You will learn about 3D transformations in the next chapter.

The translate() Method



The `translate()` method moves an element from its current position (according to the parameters given for the X-axis and the Y-axis).

The following example moves the `<div>` element 50 pixels to the right, and 100 pixels down from its current position:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    width: 300px;

    height: 100px;

    background-color: yellow;

    border: 1px solid black;
```



```
-ms-transform: translate(50px,100px); /* IE 9 */  
-webkit-transform: translate(50px,100px); /* Safari */  
transform: translate(50px,100px); /* Standard syntax */  
}  
  
</style>  
  
</head>  
  
<body>  
  
<div>  
  
The translate() method moves an element from its current position. This div  
element is moved 50 pixels to the right, and 100 pixels down from its current  
position.  
  
</div>  
  
  
  
</body>  
  
</html>
```

The rotate() Method



The `rotate()` method rotates an element clockwise or counter-clockwise according to a given degree.

The following example rotates the <div> element clockwise with 20 degrees:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    width: 300px;

    height: 100px;

    background-color: yellow;

    border: 1px solid black;

}

div#myDiv {

    -ms-transform: rotate(20deg); /* IE 9 */

    -webkit-transform: rotate(20deg); /* Safari */

    transform: rotate(20deg); /* Standard syntax */

}

</style>

</head>

<body>
```

```
<div>
```

This a normal div element.

```
</div>
```

```
<div id="myDiv">
```

The rotate() method rotates an element clockwise or counter-clockwise. This div element is rotated clockwise 20 degrees.

```
</div>
```

```
</body>
```

```
</html>
```

Using negative values will rotate the element counter-clockwise.

The following example rotates the <div> element counter-clockwise with 20 degrees:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
border: 1px solid black;  
}
```

```
div#myDiv {  
    -ms-transform: rotate(-20deg); /* IE 9 */  
    -webkit-transform: rotate(-20deg); /* Safari */  
    transform: rotate(-20deg); /* Standard syntax */  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

This a normal div element.

```
</div>
```

```
<div id="myDiv">
```

The rotate() method rotates an element clockwise or counter-clockwise. This div element is rotated counter-clockwise with 20 degrees.

```
</div>
```

```
</body>
```

</html>

The scale() Method



The `scale()` method increases or decreases the size of an element (according to the parameters given for the width and height).

The following example increases the `<div>` element to be two times of its original width, and three times of its original height:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    margin: 150px;

    width: 200px;

    height: 100px;

    background-color: yellow;

    border: 1px solid black;

    border: 1px solid black;
```

```
-ms-transform: scale(2,3); /* IE 9 */  
-webkit-transform: scale(2,3); /* Safari */  
transform: scale(2,3); /* Standard syntax */  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>The scale() method increases or decreases the size of an element.</p>
```

```
<div>
```

This div element is two times of its original width, and three times of its original height.

```
</div>
```

```
</body>
```

```
</html>
```

The following example decreases the <div> element to be half of its original width and height:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
div {
    margin: 150px;
    width: 200px;
    height: 100px;
    background-color: yellow;
    border: 1px solid black;
    border: 1px solid black;
    -ms-transform: scale(0.5,0.5); /* IE 9 */
    -webkit-transform: scale(0.5,0.5); /* Safari */
    transform: scale(0.5,0.5); /* Standard syntax */
}
</style>
</head>
<body>
```

<p>The scale() method increases or decreases the size of an element.</p>

<div>

This div element is decreased to be half of its original width and height.

</div>

```
</body>
```

```
</html>
```

The skewX() Method

The `skewX()` method skews an element along the X-axis by the given angle.

The following example skews the `<div>` element 20 degrees along the X-axis:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv {
```

```
    -ms-transform: skewX(20deg); /* IE 9 */
```

```
    -webkit-transform: skewX(20deg); /* Safari */
```

```
    transform: skewX(20deg); /* Standard syntax */
```



```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>The skewX() method skews an element along the X-axis by the given  
angle.</p>
```

```
<div>
```

```
This a normal div element.
```

```
</div>
```

```
<div id="myDiv">
```

```
This div element is skewed 20 degrees along the X-axis.
```

```
</div>
```

```
</body>
```

```
</html>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {  
    width: 300px;  
    height: 100px;  
    background-color: yellow;  
    border: 1px solid black;  
}
```

```
div#myDiv {  
    -ms-transform: skewX(20deg); /* IE 9 */  
    -webkit-transform: skewX(20deg); /* Safari */  
    transform: skewX(20deg); /* Standard syntax */  
}
```

</style>

</head>

<body>

<p>The skewX() method skews an element along the X-axis by the given angle.</p>

<div>

This a normal div element.

</div>

```
<div id="myDiv">
```

This div element is skewed 20 degrees along the X-axis.

```
</div>
```

```
</body>
```

```
</html>
```

The skewY() Method

The `skewY()` method skews an element along the Y-axis by the given angle.

The following example skews the `<div>` element 20 degrees along the Y-axis:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv {  
    -ms-transform: skewY(20deg); /* IE 9 */  
    -webkit-transform: skewY(20deg); /* Safari */  
    transform: skewY(20deg); /* Standard syntax */  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>The skewY() method skews an element along the Y-axis by the given angle.</p>

```
<div>
```

This a normal div element.

```
</div>
```

```
<div id="myDiv">
```

This div element is skewed 20 degrees along the Y-axis.

```
</div>
```

```
</body>
```

</html>

If the second parameter is not specified, it has a zero value. So, the following example skews the <div> element 20 degrees along the X-axis:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv {
```

```
    -ms-transform: skew(20deg); /* IE 9 */
```

```
    -webkit-transform: skew(20deg); /* Safari */
```

```
    transform: skew(20deg); /* Standard syntax */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>The skew() method skews an element into a given angle.</p>
```

```
<div>
```

This a normal div element.

```
</div>
```

```
<div id="myDiv">
```

This div element is skewed 20 degrees along the X-axis.

```
</div>
```

```
</body>
```

```
</html>
```

The matrix() Method



The `matrix()` method combines all the 2D transform methods into one.

The `matrix()` method takes six parameters, containing mathematic functions, which allows you to rotate, scale, move (translate), and skew elements.

The parameters are as follow:

matrix(scaleX(),skewY(),skewX(),scaleY(),translateX(),translateY()):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv1 {
```

```
    -ms-transform: matrix(1, -0.3, 0, 1, 0, 0); /* IE 9 */
```

```
    -webkit-transform: matrix(1, -0.3, 0, 1, 0, 0); /* Safari */
```

```
    transform: matrix(1, -0.3, 0, 1, 0, 0); /* Standard syntax */
```

```
}
```

```
div#myDiv2 {
```

```
    -ms-transform: matrix(1, 0, 0.5, 1, 150, 0); /* IE 9 */
```

```
-webkit-transform: matrix(1, 0, 0.5, 1, 150, 0); /* Safari */  
transform: matrix(1, 0, 0.5, 1, 150, 0); /* Standard syntax */  
}  
</style>  
</head>  
<body>
```

<p>The matrix() method combines all the 2D transform methods into one.</p>

<div>

This a normal div element.

</div>

<div id="myDiv1">

Using the matrix() method.

</div>

<div id="myDiv2">

Another use of the matrix() method.

</div>

</body>

</html>

CSS3 Transform Properties

The following table lists all the 2D transform properties:

Property	Description
transform	Applies a 2D or 3D transformation to an element
transform-origin	Allows you to change the position on transformed elements

2D Transform Methods

Function	Description
<code>matrix(<i>n,n,n,n,n,n</i>)</code>	Defines a 2D transformation, using a matrix of six values
<code>translate(<i>x,y</i>)</code>	Defines a 2D translation, moving the element along the X- and the Y-axis
<code>translateX(<i>n</i>)</code>	Defines a 2D translation, moving the element along the X-axis
<code>translateY(<i>n</i>)</code>	Defines a 2D translation, moving the element along the Y-axis
<code>scale(<i>x,y</i>)</code>	Defines a 2D scale transformation, changing the elements width and height

<code>scaleX(<i>n</i>)</code>	Defines a 2D scale transformation, changing the element's width
<code>scaleY(<i>n</i>)</code>	Defines a 2D scale transformation, changing the element's height
<code>rotate(<i>angle</i>)</code>	Defines a 2D rotation, the angle is specified in the parameter
<code>skew(<i>x-angle</i>,<i>y-angle</i>)</code>	Defines a 2D skew transformation along the X- and the Y-axis
<code>skewX(<i>angle</i>)</code>	Defines a 2D skew transformation along the X-axis
<code>skewY(<i>angle</i>)</code>	Defines a 2D skew transformation along the Y-axis

CSS3 3D Transforms

CSS3 allows you to format your elements using 3D transformations.

Mouse over the elements below to see the difference between a 2D and a 3D transformation:

2D rotate

3D rotate

Browser Support for 3D Transforms

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by `-webkit-`, `-moz-`, or `-o-` specify the first version that worked with a prefix.

Property					
transform	36.0 12.0 -webkit-	10.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-
transform-origin (three-value syntax)	36.0 12.0 -webkit-	10.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-
transform-style	36.0 12.0 -webkit-	11.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-
perspective	36.0 12.0 -webkit-	10.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-
perspective-origin	36.0 12.0 -webkit-	10.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-
backface-visibility	36.0 12.0 -webkit-	10.0	16.0 10.0 -moz-	9.0 4.0 -webkit-	23.0 15.0 -webkit-

CSS3 3D Transforms

In this chapter you will learn about the following 3D transformation methods:

- `rotateX()`
- `rotateY()`
- `rotateZ()`

The rotateX() Method



The `rotateX()` method rotates an element around its X-axis at a given degree:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv {
```

```
    -webkit-transform: rotateX(150deg); /* Safari */
```

```
    transform: rotateX(150deg); /* Standard syntax */
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

```
This a normal div element.
```

```
</div>
```

```
<div id="myDiv">
```

The rotateX() method rotates an element around its X-axis at a given degree. This div element is rotated 150 degrees.

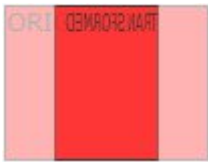
```
</div>
```

<p>Note: Internet Explorer 9 (and earlier versions) does not support the rotateX() method.</p>

```
</body>
```

```
</html>
```

The rotateY() Method



The `rotateY()` method rotates an element around its Y-axis at a given degree:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
width: 300px;
height: 100px;
background-color: yellow;
border: 1px solid black;
}
```

```
div#myDiv {
    -webkit-transform: rotateY(150deg); /* Safari */
    transform: rotateY(150deg); /* Standard syntax */
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

This a normal div element.

```
</div>
```

```
<div id="myDiv">
```

The rotateY() method rotates an element around its Y-axis at a given degree. This div element is rotated 150 degrees.

```
</div>
```

<p>Note: Internet Explorer 9 (and earlier versions) does not support the rotateY() method.</p>

</body>

</html>

The rotateZ() Method

The `rotateZ()` method rotates an element around its Z-axis at a given degree:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    background-color: yellow;
```

```
    border: 1px solid black;
```

```
}
```

```
div#myDiv {
```

```
    -webkit-transform: rotateZ(90deg); /* Safari */
```

```
    transform: rotateZ(90deg); /* Standard syntax */
}

</style>

</head>

<body>

<div>

This a normal div element.

</div>

<div id="myDiv">

The rotateZ() method rotates an element around its Z-axis at a given degree. This
div element is rotated 90 degrees.

</div>

<p><b>Note:</b> Internet Explorer 9 (and earlier versions) does not support the
rotateZ() method.</p>

</body>

</html>
```

CSS3 Transform Properties

The following table lists all the 3D transform properties:

Property	Description
transform	Applies a 2D or 3D transformation to an element
transform-origin	Allows you to change the position on transformed elements
transform-style	Specifies how nested elements are rendered in 3D space
perspective	Specifies the perspective on how 3D elements are viewed
perspective-origin	Specifies the bottom position of 3D elements
backface-visibility	Defines whether or not an element should be visible when not facing the screen

3D Transform Methods

Function	Description
matrix3d (n,n,n,n,n,n,n,n,n,n,n,n,n,n,n,n)	Defines a 3D transformation, using a 4x4 matrix of 16 values
translate3d(x,y,z)	Defines a 3D translation
translateX(x)	Defines a 3D translation, using only the value for the X-axis

<code>translateY(y)</code>	Defines a 3D translation, using only the value for the Y-axis
<code>translateZ(z)</code>	Defines a 3D translation, using only the value for the Z-axis
<code>scale3d(x,y,z)</code>	Defines a 3D scale transformation
<code>scaleX(x)</code>	Defines a 3D scale transformation by giving a value for the X-axis
<code>scaleY(y)</code>	Defines a 3D scale transformation by giving a value for the Y-axis
<code>scaleZ(z)</code>	Defines a 3D scale transformation by giving a value for the Z-axis
<code>rotate3d(x,y,z,angle)</code>	Defines a 3D rotation
<code>rotateX(<i>angle</i>)</code>	Defines a 3D rotation along the X-axis
<code>rotateY(<i>angle</i>)</code>	Defines a 3D rotation along the Y-axis
<code>rotateZ(<i>angle</i>)</code>	Defines a 3D rotation along the Z-axis
<code>perspective(<i>n</i>)</code>	Defines a perspective view for a 3D

transformed element

CSS3 Animations

[< Previous](#) [Next >](#)

CSS3 Animations

CSS3 animations allows animation of most HTML elements without using JavaScript or Flash!

CSS3 Animation

Browser Support for Animations

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit-, -moz-, or -o- specify the first version that worked with a prefix.

Property					
@keyframes	43.0 4.0 -webkit-	10.0	16.0 5.0 -moz-	9.0 4.0 -webkit-	30.0 15.0 -webkit- 12.0 -o-
animation	43.0 4.0 -webkit-	10.0	16.0 5.0 -moz-	9.0 4.0 -webkit-	30.0 15.0 -webkit- 12.0 -o-

What are CSS3 Animations?

An animation lets an element gradually change from one style to another.

You can change as many CSS properties you want, as many times you want.

To use CSS3 animation, you must first specify some keyframes for the animation.

Keyframes hold what styles the element will have at certain times.

The @keyframes Rule

When you specify CSS styles inside the `@keyframes` rule, the animation will gradually change from the current style to the new style at certain times.

To get an animation to work, you must bind the animation to an element.

The following example binds the "example" animation to the `<div>` element. The animation will last for 4 seconds, and it will gradually change the background-color of the `<div>` element from "red" to "yellow":

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    width: 100px;

    height: 100px;

    background-color: red;

    -webkit-animation-name: example; /* Chrome, Safari, Opera */

    -webkit-animation-duration: 4s; /* Chrome, Safari, Opera */
```

```
    animation-name: example;  
    animation-duration: 4s;  
}
```

```
/* Chrome, Safari, Opera */  
@-webkit-keyframes example {  
    from {background-color: red;}  
    to {background-color: yellow;}  
}
```

```
/* Standard syntax */  
@keyframes example {  
    from {background-color: red;}  
    to {background-color: yellow;}  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>Note: This example does not work in Internet Explorer 9 and earlier versions.</p>

```
<div></div>
```

```
<p><b>Note:</b> When an animation is finished, it changes back to its original style.</p>
```

```
</body>
```

```
</html>
```

Note: If the `animation-duration` property is not specified, the animation will have no effect, because the default value is 0.

In the example above we have specified when the style will change by using the keywords "from" and "to" (which represents 0% (start) and 100% (complete)).

It is also possible to use percent. By using percent, you can add as many style changes as you like.

The following example will change the background-color of the `<div>` element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
-webkit-animation-name: example; /* Chrome, Safari, Opera */  
-webkit-animation-duration: 4s; /* Chrome, Safari, Opera */  
animation-name: example;  
animation-duration: 4s;  
}
```

```
/* Chrome, Safari, Opera */  
@-webkit-keyframes example {  
    0% {background-color: red;}  
    25% {background-color: yellow;}  
    50% {background-color: blue;}  
    100% {background-color: green;}  
}
```

```
/* Standard syntax */  
@keyframes example {  
    0% {background-color: red;}  
    25% {background-color: yellow;}  
    50% {background-color: blue;}  
    100% {background-color: green;}  
}  
  
</style>
```



```
</head>
```

```
<body>
```

```
<p><b>Note:</b> This example does not work in Internet Explorer 9 and earlier  
versions.</p>
```

```
<div></div>
```

```
</body>
```

```
</html>
```

The following example will change both the background-color and the position of the <div> element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
position: relative;

-webkit-animation-name: example; /* Chrome, Safari, Opera */
-webkit-animation-duration: 4s; /* Chrome, Safari, Opera */
animation-name: example;
animation-duration: 4s;
}
```

```
/* Chrome, Safari, Opera */
@-webkit-keyframes example {
    0% {background-color:red; left:0px; top:0px;}
    25% {background-color:yellow; left:200px; top:0px;}
    50% {background-color:blue; left:200px; top:200px;}
    75% {background-color:green; left:0px; top:200px;}
    100% {background-color:red; left:0px; top:0px;}
}
```

```
/* Standard syntax */
@keyframes example {
    0% {background-color:red; left:0px; top:0px;}
    25% {background-color:yellow; left:200px; top:0px;}
    50% {background-color:blue; left:200px; top:200px;}
    75% {background-color:green; left:0px; top:200px;}
}
```

```
100% {background-color:red; left:0px; top:0px;}  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>Note: This example does not work in Internet Explorer 9 and earlier versions.</p>

```
<div></div>
```

```
</body>
```

```
</html>
```

Delay an Animation

The `animation-delay` property specifies a delay for the start of an animation.

The following example has a 2 seconds delay before starting the animation:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
width: 100px;
height: 100px;
background-color: red;
position: relative;
-webkit-animation-name: example; /* Chrome, Safari, Opera */
-webkit-animation-duration: 4s; /* Chrome, Safari, Opera */
-webkit-animation-delay: 2s; /* Chrome, Safari, Opera */
animation-name: example;
animation-duration: 4s;
animation-delay: 2s;
}
```

```
/* Chrome, Safari, Opera */
```

```
@-webkit-keyframes example {
  0% {background-color:red; left:0px; top:0px;}
  25% {background-color:yellow; left:200px; top:0px;}
  50% {background-color:blue; left:200px; top:200px;}
  75% {background-color:green; left:0px; top:200px;}
  100% {background-color:red; left:0px; top:0px;}
}
```

```
/* Standard syntax */
```

```
@keyframes example {  
  0% {background-color:red; left:0px; top:0px;}  
  25% {background-color:yellow; left:200px; top:0px;}  
  50% {background-color:blue; left:200px; top:200px;}  
  75% {background-color:green; left:0px; top:200px;}  
  100% {background-color:red; left:0px; top:0px;}  
}  
  
</style>  
  
</head>  
  
<body>
```

<p>Note: This example does not work in Internet Explorer 9 and earlier versions.</p>

<div></div>

</body>

</html>

Set How Many Times an Animation Should Run

The `animation-iteration-count` property specifies the number of times an animation should run.

The following example will run the animation 3 times before it stops:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    width: 100px;

    height: 100px;

    background-color: red;

    position: relative;

    -webkit-animation-name: example; /* Chrome, Safari, Opera */

    -webkit-animation-duration: 4s; /* Chrome, Safari, Opera */

    -webkit-animation-iteration-count: 3; /* Chrome, Safari, Opera */

    animation-name: example;

    animation-duration: 4s;

    animation-iteration-count: 3;

}

/* Chrome, Safari, Opera */

@-webkit-keyframes example {

    0% {background-color:red; left:0px; top:0px;}
```

```
25% {background-color:yellow; left:200px; top:0px;}  
50% {background-color:blue; left:200px; top:200px;}  
75% {background-color:green; left:0px; top:200px;}  
100% {background-color:red; left:0px; top:0px;}  
}
```

```
/* Standard syntax */
```

```
@keyframes example {  
  0% {background-color:red; left:0px; top:0px;}  
  25% {background-color:yellow; left:200px; top:0px;}  
  50% {background-color:blue; left:200px; top:200px;}  
  75% {background-color:green; left:0px; top:200px;}  
  100% {background-color:red; left:0px; top:0px;}  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

Note: This example does not work in Internet Explorer 9 and earlier versions.

```
<div></div>
```

```
</body>
```

```
</html>
```

The following example uses the value "infinite" to make the animation continue for ever:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
    position: relative;
```

```
    -webkit-animation-name: example; /* Chrome, Safari, Opera */
```

```
    -webkit-animation-duration: 4s; /* Chrome, Safari, Opera */
```

```
    -webkit-animation-iteration-count: infinite; /* Chrome, Safari, Opera */
```

```
    animation-name: example;
```

```
    animation-duration: 4s;
```

```
    animation-iteration-count: infinite;
```

```
}
```



```
/* Chrome, Safari, Opera */
```

```
@-webkit-keyframes example {
```

```
    0% {background-color:red; left:0px; top:0px;}
```

```
    25% {background-color:yellow; left:200px; top:0px;}
```

```
    50% {background-color:blue; left:200px; top:200px;}
```

```
    75% {background-color:green; left:0px; top:200px;}
```

```
    100% {background-color:red; left:0px; top:0px;}
```

```
}
```

```
/* Standard syntax */
```

```
@keyframes example {
```

```
    0% {background-color:red; left:0px; top:0px;}
```

```
    25% {background-color:yellow; left:200px; top:0px;}
```

```
    50% {background-color:blue; left:200px; top:200px;}
```

```
    75% {background-color:green; left:0px; top:200px;}
```

```
    100% {background-color:red; left:0px; top:0px;}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>Note: This example does not work in Internet Explorer 9 and earlier versions.</p>

<div></div>

</body>

</html>

Run Animation in Reverse Direction or Alternate Cycles

The `animation-direction` property is used to let an animation run in reverse direction or alternate cycles.

The following example will run the animation in reverse direction:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
    position: relative;
```

```
-webkit-animation-name: example; /* Chrome, Safari, Opera */  
-webkit-animation-duration: 4s; /* Chrome, Safari, Opera */  
-webkit-animation-iteration-count: 3; /* Chrome, Safari, Opera */  
-webkit-animation-direction: reverse; /* Chrome, Safari, Opera */  
animation-name: example;  
animation-duration: 4s;  
animation-iteration-count: 3;  
animation-direction: reverse;  
}
```

```
/* Chrome, Safari, Opera */  
@-webkit-keyframes example {  
    0% {background-color:red; left:0px; top:0px;}  
    25% {background-color:yellow; left:200px; top:0px;}  
    50% {background-color:blue; left:200px; top:200px;}  
    75% {background-color:green; left:0px; top:200px;}  
    100% {background-color:red; left:0px; top:0px;}  
}
```

```
/* Standard syntax */  
@keyframes example {  
    0% {background-color:red; left:0px; top:0px;}  
    25% {background-color:yellow; left:200px; top:0px;}  
    50% {background-color:blue; left:200px; top:200px;}  
    75% {background-color:green; left:0px; top:200px;}  
    100% {background-color:red; left:0px; top:0px;}  
}
```

```
25% {background-color:yellow; left:200px; top:0px;}  
50% {background-color:blue; left:200px; top:200px;}  
75% {background-color:green; left:0px; top:200px;}  
100% {background-color:red; left:0px; top:0px;}  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

<p>Note: This example does not work in Internet Explorer 9 and earlier versions.</p>

```
<div></div>
```

```
</body>
```

```
</html>
```

The following example uses the value "alternate" to make the animation first run forward, then backward, then forward:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {  
    width: 100px;  
    height: 100px;  
    background-color: red;  
    position: relative;  
  
    -webkit-animation-name: example; /* Chrome, Safari, Opera */  
    -webkit-animation-duration: 4s; /* Chrome, Safari, Opera */  
    -webkit-animation-iteration-count: 3; /* Chrome, Safari, Opera */  
    -webkit-animation-direction: alternate; /* Chrome, Safari, Opera */  
  
    animation-name: example;  
    animation-duration: 4s;  
    animation-iteration-count: 3;  
    animation-direction: alternate;  
}
```

```
/* Chrome, Safari, Opera */
```

```
@-webkit-keyframes example {  
    0% {background-color:red; left:0px; top:0px;}  
    25% {background-color:yellow; left:200px; top:0px;}  
    50% {background-color:blue; left:200px; top:200px;}  
    75% {background-color:green; left:0px; top:200px;}  
    100% {background-color:red; left:0px; top:0px;}
```

```
}
```

```
/* Standard syntax */
```

```
@keyframes example {
```

```
  0% {background-color:red; left:0px; top:0px;}
```

```
  25% {background-color:yellow; left:200px; top:0px;}
```

```
  50% {background-color:blue; left:200px; top:200px;}
```

```
  75% {background-color:green; left:0px; top:200px;}
```

```
  100% {background-color:red; left:0px; top:0px;}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

Note: This example does not work in Internet Explorer 9 and earlier versions.

```
<div></div>
```

```
</body>
```

```
</html>
```

Specify the Speed Curve of the Animation

The `animation-timing-function` property specifies the speed curve of the animation.

The animation-timing-function property can have the following values:

- `ease` - specifies an animation with a slow start, then fast, then end slowly (this is default)
- `linear` - specifies an animation with the same speed from start to end
- `ease-in` - specifies an animation with a slow start
- `ease-out` - specifies an animation with a slow end
- `ease-in-out` - specifies an animation with a slow start and end
- `cubic-bezier(n,n,n,n)` - lets you define your own values in a cubic-bezier function

The following example shows the some of the different speed curves that can be used:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    width: 100px;

    height: 50px;

    background-color: red;

    font-weight: bold;

    position: relative;
```

```
-webkit-animation: mymove 5s infinite; /* Chrome, Safari, Opera */  
animation: mymove 5s infinite;  
}
```

```
/* Chrome, Safari, Opera */  
  
#div1 {-webkit-animation-timing-function: linear;}  
#div2 {-webkit-animation-timing-function: ease;}  
#div3 {-webkit-animation-timing-function: ease-in;}  
#div4 {-webkit-animation-timing-function: ease-out;}  
#div5 {-webkit-animation-timing-function: ease-in-out;}
```

```
/* Standard syntax */  
  
#div1 {animation-timing-function: linear;}  
#div2 {animation-timing-function: ease;}  
#div3 {animation-timing-function: ease-in;}  
#div4 {animation-timing-function: ease-out;}  
#div5 {animation-timing-function: ease-in-out;}
```

```
/* Chrome, Safari, Opera */  
  
@-webkit-keyframes mymove {  
    from {left: 0px;}  
    to {left: 300px;}
```



```
}
```

```
/* Standard syntax */
```

```
@keyframes mymove {
```

```
    from {left: 0px;}
```

```
    to {left: 300px;}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

Note: The animation-timing-funtion property is not supported in Internet Explorer 9 and earlier versions.

```
<div id="div1">linear</div>
```

```
<div id="div2">ease</div>
```

```
<div id="div3">ease-in</div>
```

```
<div id="div4">ease-out</div>
```

```
<div id="div5">ease-in-out</div>
```

```
</body>
```

```
</html>
```

Animation Shorthand Property

The example below uses six of the animation properties:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
    position: relative;
```

```
    /* Chrome, Safari, Opera */
```

```
    -webkit-animation-name: example;
```

```
    -webkit-animation-duration: 5s;
```

```
    -webkit-animation-timing-function: linear;
```

```
    -webkit-animation-delay: 2s;
```

```
    -webkit-animation-iteration-count: infinite;
```

```
    -webkit-animation-direction: alternate;
```

```
    /* Standard syntax */
```

```
    animation-name: example;
```

```
    animation-duration: 5s;
```

```
animation-timing-function: linear;

animation-delay: 2s;

animation-iteration-count: infinite;

animation-direction: alternate;

}
```

```
/* Chrome, Safari, Opera */
```

```
@-webkit-keyframes example {

  0% {background-color:red; left:0px; top:0px;}

  25% {background-color:yellow; left:200px; top:0px;}

  50% {background-color:blue; left:200px; top:200px;}

  75% {background-color:green; left:0px; top:200px;}

  100% {background-color:red; left:0px; top:0px;}

}
```

```
/* Standard syntax */
```

```
@keyframes example {

  0% {background-color:red; left:0px; top:0px;}

  25% {background-color:yellow; left:200px; top:0px;}

  50% {background-color:blue; left:200px; top:200px;}

  75% {background-color:green; left:0px; top:200px;}

  100% {background-color:red; left:0px; top:0px;}

}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><b>Note:</b> This example does not work in Internet Explorer 9 and earlier  
versions.</p>
```

```
<div></div>
```

```
</body>
```

```
</html>
```

The same animation effect as above can be achieved by using the shorthand `animation` property:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    background-color: red;
```

```
position: relative;

-webkit-animation: myfirst 5s linear 2s infinite alternate; /* Chrome, Safari,
Opera */

animation: myfirst 5s linear 2s infinite alternate;

}
```

```
/* Chrome, Safari, Opera */
```

```
@-webkit-keyframes myfirst {

    0% {background-color:red; left:0px; top:0px;}

    25% {background-color:yellow; left:200px; top:0px;}

    50% {background-color:blue; left:200px; top:200px;}

    75% {background-color:green; left:0px; top:200px;}

    100% {background-color:red; left:0px; top:0px;}

}
```

```
/* Standard syntax */
```

```
@keyframes myfirst {

    0% {background-color:red; left:0px; top:0px;}

    25% {background-color:yellow; left:200px; top:0px;}

    50% {background-color:blue; left:200px; top:200px;}

    75% {background-color:green; left:0px; top:200px;}

    100% {background-color:red; left:0px; top:0px;}

}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><b>Note:</b> This example does not work in Internet Explorer 9 and earlier  
versions.</p>
```

```
<div></div>
```

```
</body>
```

```
</html>
```

CSS Images

[< Previous](#)[Next >](#)

Learn how to style images using CSS.

Rounded Images

Use the `border-radius` property to create rounded images:

Example

Rounded Image:

```
<!DOCTYPE html>

<html>

<head>

<style>

img {

    border-radius: 8px;

}

</style>

</head>

<body>


<h2>Rounded Images</h2>

<p>Use the border-radius property to create rounded images:</p>





</body>

</html>
```

Example

Circled Image:

```
<!DOCTYPE html>

<html>

<head>

<style>

img {
    border-radius: 50%;
}

</style>

</head>

<body>

<h2>Rounded Images</h2>

<p>Use the border-radius property to create circled images:</p>



</body>

</html>
```

Thumbnail Images

Use the **border** property to create thumbnail images.

Thumbnail Image:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
img {
```

```
    border: 1px solid #ddd;
```

```
    border-radius: 4px;
```

```
    padding: 5px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Thumbnail Images</h2>
```

```
<p>Use the border property to create thumbnail images:</p>
```

```

```

```
</body>
```

```
</html>
```

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

a {

    display: inline-block;

    border: 1px solid #ddd;

    border-radius: 4px;

    padding: 5px;

    transition: 0.3s;

}

a:hover {

    box-shadow: 0 0 2px 1px rgba(0, 140, 186, 0.5);

}

</style>

</head>

<body>

<h2>Thumbnail Image as Link</h2>
```

<p>Use the border property to create thumbnail images. Wrap an anchor around the image to use it as a link.</p>

<p>Hover over the image and click on it to see the effect.</p>

</body>

</html>

Responsive Images

Responsive images will automatically adjust to fit the size of the screen.

Resize the browser window to see the effect:

If you want an image to scale down if it has to, but never scale up to be larger than its original size, add the following:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
img {
```

```
    max-width: 100%;
```

```
    height: auto;
}

</style>

</head>

<body>

<h2>Responsive Images</h2>

<p>Responsive images will automatically adjust to fit the size of the screen.</p>

<p>Resize the browser window to see the effect:</p>



</body>

</html>
```

Tip: Read more about Responsive Web Design in our [CSS RWD Tutorial](#).

Center an Image

To center an image within the page, use `margin: auto;` and make it into a **block** element:

```
<!DOCTYPE html>

<html>

<head>
```

```
<style>
img {
    display: block;
    margin: auto;
}
</style>
</head>
<body>



</body>
</html>
```

Image Text

How to position text in an image:

Example



Bottom Left

Top Left

Top Right

Bottom Right

Centered

Try it Yourself:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.container {
```

```
  position: relative;
```

```
}
```

```
.topleft {  
    position: absolute;  
    top: 8px;  
    left: 16px;  
    font-size: 18px;  
}
```

```
img {  
    width: 100%;  
    height: auto;  
    opacity: 0.3;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Image Text</h2>
```

```
<p>Add some text to an image in the top left corner:</p>
```

```
<div class="container">
```

```
  
```

```
<div class="topleft">Top Left</div>  
</div>
```

```
</body>
```

```
</html>
```

Example 2

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.container {  
    position: relative;  
}
```

```
.topright {  
    position: absolute;  
    top: 8px;  
    right: 16px;  
    font-size: 18px;  
}
```

```
img {
```



```
width: 100%;  
height: auto;  
opacity: 0.3;  
}  
</style>  
</head>  
<body>  
  
<h2>Image Text</h2>  
  
<p>Add some text to an image in the top right corner:</p>  
  
<div class="container">  
    
  <div class="topright">Top Right</div>  
</div>  
  
</body>  
</html>
```

Example 3

```
<!DOCTYPE html>  
  
<html>  
  
<head>
```

```
<style>

.container {
    position: relative;
}

.bottomleft {
    position: absolute;
    bottom: 8px;
    left: 16px;
    font-size: 18px;
}

img {
    width: 100%;
    height: auto;
    opacity: 0.3;
}

</style>

</head>

<body>

<h2>Image Text</h2>
```

<p>Add some text to an image in the bottom left corner:</p>

<div class="container">

<div class="bottomleft">Bottom Left</div>

</div>

</body>

</html>

Example4

<!DOCTYPE html>

<html>

<head>

<style>

.container {

position: relative;

}

.bottomleft {

position: absolute;

bottom: 8px;

left: 16px;

```
    font-size: 18px;
}
```

```
img {
    width: 100%;
    height: auto;
    opacity: 0.3;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Image Text</h2>
```

```
<p>Add some text to an image in the bottom left corner:</p>
```

```
<div class="container">
```

```
    
```

```
    <div class="bottomleft">Bottom Left</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

Example 4

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.container {
```

```
    position: relative;
```

```
}
```

```
.bottomright {
```

```
    position: absolute;
```

```
    bottom: 8px;
```

```
    right: 16px;
```

```
    font-size: 18px;
```

```
}
```

```
img {
```

```
    width: 100%;
```

```
    height: auto;
```

```
    opacity: 0.3;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Image Text</h2>
```

```
<p>Add some text to an image in the bottom right corner:</p>
```

```
<div class="container">
```

```
  
```

```
  <div class="bottomright">Bottom Right</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

Example 5

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.container {
```

```
  position: relative;
```

```
}
```

```
.center {  
    position: absolute;  
    top: 50%;  
    left: 50%;  
    transform: translate(-50%, -50%);  
    font-size: 18px;  
}
```

```
img {  
    width: 100%;  
    height: auto;  
    opacity: 0.3;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Image Text</h2>
```

```
<p>Center text in image:</p>
```

```
<div class="container">
```

```

```

```
<div class="center">Centered</div>

</div>
```

```
</body>
```

```
</html>
```

Polaroid Images / Cards

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body {margin:25px;}
```

```
div.polaroid {
```

```
  width: 80%;
```

```
  background-color: white;
```

```
  box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.19);
```

```
  margin-bottom: 25px;
```

```
}
```

```
div.container {
```

```
  text-align: center;
```

```
  padding: 10px 20px;
```



```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Responsive Polaroid Images / Cards</h2>
```

```
<div class="polaroid">
```

```

```

```
<div class="container">
```

```
<p>The Troll's tongue in Hardanger, Norway</p>
```

```
</div>
```

```
</div>
```

```
<div class="polaroid">
```

```

```

```
<div class="container">
```

```
<p>Northern Lights in Norway</p>
```

```
</div>
```

```
</div>
```

```
</body>
```

</html>

Image Filters

The CSS **filter** property adds visual effects (like blur and saturation) to an element.

Note: The filter property is not supported in Internet Explorer, Edge 12, or Safari 5.1 and earlier.

Example

Change the color of all images to black and white (100% gray):

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
img {
```

```
    width: 33%;
```

```
    height: auto;
```

```
    float: left;
```

```
    max-width: 235px;
```

```
}
```

```
.blur {-webkit-filter: blur(4px);filter: blur(4px);}
```

```
.brightness {-webkit-filter: brightness(250%);filter: brightness(250%);}
```

```
.contrast {-webkit-filter: contrast(180%);filter: contrast(180%);}
```

```
.grayscale {-webkit-filter: grayscale(100%);filter: grayscale(100%);}

.huerotate {-webkit-filter: hue-rotate(180deg);filter: hue-rotate(180deg);}

.invert {-webkit-filter: invert(100%);filter: invert(100%);}

.opacity {-webkit-filter: opacity(50%);filter: opacity(50%);}

.saturate {-webkit-filter: saturate(7); filter: saturate(7);}

.sepia {-webkit-filter: sepia(100%);filter: sepia(100%);}

.shadow {-webkit-filter: drop-shadow(8px 8px 10px green);filter: drop-
shadow(8px 8px 10px green);}

</style>

</head>

<body>
```

<p>Note: The filter property is not supported in Internet Explorer, Edge 12, or Safari 5.1 and earlier.</p>

```









```

```

```

```

```

```

```

```

```

```

```

```

```

```
</body>
```

```
</html>
```

Responsive Image Gallery

CSS can be used to create image galleries. This example use media queries to re-arrange the images on different screen sizes. Resize the browser window to see the effect:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div.img {
```

```
    border: 1px solid #ccc;
```

```
}
```

```
div.img:hover {  
    border: 1px solid #777;  
}
```

```
div.img img {  
    width: 100%;  
    height: auto;  
}
```

```
div.desc {  
    padding: 15px;  
    text-align: center;  
}
```

```
* {  
    box-sizing: border-box;  
}
```

```
.responsive {  
    padding: 0 6px;
```

```
float: left;

width: 24.99999%;

}
```

```
@media only screen and (max-width: 700px){

    .responsive {

        width: 49.99999%;

        margin: 6px 0;

    }

}
```

```
@media only screen and (max-width: 500px){

    .responsive {

        width: 100%;

    }

}
```

```
.clearfix:after {

    content: "";

    display: table;

    clear: both;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Responsive Image Gallery</h2>
```

```
<h4>Resize the browser window to see the effect.</h4>
```

```
<div class="responsive">
```

```
<div class="img">
```

```
<a target="_blank" href="img_fjords.jpg">
```

```

```

```
</a>
```

```
<div class="desc">Add a description of the image here</div>
```

```
</div>
```

```
</div>
```

```
<div class="responsive">
```

```
<div class="img">
```

```
<a target="_blank" href="img_forest.jpg">
```

```

```


<div class="desc">Add a description of the image here</div>

</div>

</div>

<div class="responsive">

<div class="img">

<div class="desc">Add a description of the image here</div>

</div>

</div>

<div class="responsive">

<div class="img">

<div class="desc">Add a description of the image here</div>

</div>

</div>


```
<div class="clearfix"></div>
```

```
<div style="padding:6px;">
```

```
<p>This example use media queries to re-arrange the images on different screen sizes: for screens larger than 700px wide, it will show four images side by side, for screens smaller than 700px, it will show two images side by side. For screens smaller than 500px, the images will stack vertically (100%).</p>
```

```
<p>You will learn more about media queries and responsive web design later in our CSS Tutorial.</p>
```

```
</div>
```

```
</body>
```

```
</html>
```

Tip: Read more about Responsive Web Design in our [CSS RWD Tutorial](#).

Image Modal (Advanced)

This is an example to demonstrate how CSS and JavaScript can work together.

First, use CSS to create a modal window (dialog box), and hide it by default.

Then, use a JavaScript to show the modal window and to display the image inside the modal, when a user clicks on the image:



Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#myImg {
```

```
    border-radius: 5px;
```

```
    cursor: pointer;
```

```
    transition: 0.3s;
```

```
}
```

```
#myImg:hover {opacity: 0.7;}
```

```
/* The Modal (background) */
```

```
.modal {
```

```
    display: none; /* Hidden by default */
```

```
position: fixed; /* Stay in place */
z-index: 1; /* Sit on top */
padding-top: 100px; /* Location of the box */
left: 0;
top: 0;
width: 100%; /* Full width */
height: 100%; /* Full height */
overflow: auto; /* Enable scroll if needed */
background-color: rgb(0,0,0); /* Fallback color */
background-color: rgba(0,0,0,0.9); /* Black w/ opacity */
}
```

```
/* Modal Content (image) */
```

```
.modal-content {
    margin: auto;
    display: block;
    width: 80%;
    max-width: 700px;
}
```

```
/* Caption of Modal Image */
```

```
#caption {
```

```
margin: auto;
display: block;
width: 80%;
max-width: 700px;
text-align: center;
color: #ccc;
padding: 10px 0;
height: 150px;
}
```

```
/* Add Animation */
```

```
.modal-content, #caption {
  -webkit-animation-name: zoom;
  -webkit-animation-duration: 0.6s;
  animation-name: zoom;
  animation-duration: 0.6s;
}
```

```
@-webkit-keyframes zoom {
  from {-webkit-transform: scale(0)}
  to {-webkit-transform: scale(1)}
}
```

```
@keyframes zoom {  
  from {transform: scale(0.1)}  
  to {transform: scale(1)}  
}
```

```
/* The Close Button */
```

```
.close {  
  position: absolute;  
  top: 15px;  
  right: 35px;  
  color: #f1f1f1;  
  font-size: 40px;  
  font-weight: bold;  
  transition: 0.3s;  
}
```

```
.close:hover,  
.close:focus {  
  color: #bbb;  
  text-decoration: none;  
  cursor: pointer;
```

```
}
```

```
/* 100% Image Width on Smaller Screens */
```

```
@media only screen and (max-width: 700px){
```

```
  .modal-content {
```

```
    width: 100%;
```

```
  }
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Image Modal</h2>
```

```
<p>In this example, we use CSS to create a modal (dialog box) that is hidden by default.</p>
```

```
<p>We use JavaScript to trigger the modal and to display the current image inside the modal when it is clicked on. Also note that we use the value from the image's "alt" attribute as an image caption text inside the modal.</p>
```

```
<p>Don't worry if you do not understand the code right away. When you are done with CSS, go to our JavaScript Tutorial to learn more.</p>
```

```

```

```
<!-- The Modal -->
```

```
<div id="myModal" class="modal">
```

```
  <span class="close">x</span>
```

```
  <img class="modal-content" id="img01">
```

```
  <div id="caption"></div>
```

```
</div>
```

```
<script>
```

```
// Get the modal
```

```
var modal = document.getElementById('myModal');
```

```
// Get the image and insert it inside the modal - use its "alt" text as a caption
```

```
var img = document.getElementById('myImg');
```

```
var modalImg = document.getElementById("img01");
```

```
var captionText = document.getElementById("caption");
```

```
img.onclick = function(){
```

```
    modal.style.display = "block";
```

```
    modalImg.src = this.src;
```

```
    captionText.innerHTML = this.alt;
```

```
}
```

```
// Get the <span> element that closes the modal

var span = document.getElementsByClassName("close")[0];


// When the user clicks on <span> (x), close the modal

span.onclick = function() {

    modal.style.display = "none";

}

</script>


</body>

</html>
```

CSS Buttons

[< Previous](#) [Next >](#)

Learn how to style buttons using CSS.

Basic Button Styling

Default Button CSS Button

```
<!DOCTYPE html>

<html>

<head>
```



```
<style>
.button {
    background-color: #4CAF50;
    border: none;
    color: white;
    padding: 15px 32px;
    text-align: center;
    text-decoration: none;
    display: inline-block;
    font-size: 16px;
    margin: 4px 2px;
    cursor: pointer;
}
</style>
</head>
<body>

<h2>CSS Buttons</h2>

<button>Default Button</button>

<a href="#" class="button">Link Button</a>

<button class="button">Button</button>
```

```
<input type="button" class="button" value="Input Button">
```

```
</body>
```

```
</html>
```

Button Colors

Green Blue Red Gray Black

Use the `background-color` property to change the background color of a button:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
    color: white;
```

```
    padding: 15px 32px;
```

```
    text-align: center;
```

```
    text-decoration: none;
```

```
    display: inline-block;
```

```
    font-size: 16px;
```

```
    margin: 4px 2px;
```

```
    cursor: pointer;
```

```
}
```

```
.button2 {background-color: #008CBA;} /* Blue */
```

```
.button3 {background-color: #f44336;} /* Red */
```

```
.button4 {background-color: #e7e7e7; color: black;} /* Gray */
```

```
.button5 {background-color: #555555;} /* Black */
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Button Colors</h2>
```

```
<p>Change the background color of a button with the background-color  
property:</p>
```

```
<button class="button">Green</button>
```

```
<button class="button button2">Blue</button>
```

```
<button class="button button3">Red</button>
```

```
<button class="button button4">Gray</button>
```

```
<button class="button button5">Black</button>
```

```
</body>
```

```
</html>
```

Button Sizes

10px 12px 16px 20px 24px

Use the `font-size` property to change the font size of a button:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
    color: white;
```

```
    padding: 15px 32px;
```

```
    text-align: center;
```

```
    text-decoration: none;
```

```
    display: inline-block;
```

```
    margin: 4px 2px;
```

```
    cursor: pointer;
```

```
}
```

```
.button1 {font-size: 10px;}
```

```
.button2 {font-size: 12px;}
```

```
.button3 {font-size: 16px;}
```

```
.button4 {font-size: 20px;}
```

```
.button5 {font-size: 24px;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Button Sizes</h2>
```

```
<p>Change the font size of a button with the font-size property:</p>
```

```
<button class="button button1">10px</button>
```

```
<button class="button button2">12px</button>
```

```
<button class="button button3">16px</button>
```

```
<button class="button button4">20px</button>
```

```
<button class="button button5">24px</button>
```

```
</body>
```

```
</html>
```

Use the **padding** property to change the padding of a button:

10px 24px 12px 28px 14px 40px 32px 16px

Example

```
<!DOCTYPE html>
```

```
<html>

<head>

<style>

.button {

    background-color: #4CAF50; /* Green */

    border: none;

    color: white;

    text-align: center;

    text-decoration: none;

    display: inline-block;

    font-size: 16px;

    margin: 4px 2px;

    cursor: pointer;

}


.button1 {padding: 10px 24px;}

.button2 {padding: 12px 28px;}

.button3 {padding: 14px 40px;}

.button4 {padding: 32px 16px;}

.button5 {padding: 16px;}

</style>

</head>
```

```
<body>
```

```
<h2>Button Sizes</h2>
```

```
<p>Change the padding of a button with the padding property:</p>
```

```
<button class="button button1">10px 24px</button>
```

```
<button class="button button2">12px 28px</button>
```

```
<button class="button button3">14px 40px</button>
```

```
<button class="button button4">32px 16px</button>
```

```
<button class="button button5">16px</button>
```

```
</body>
```

```
</html>
```

Rounded Buttons

2px 4px 8px 12px 50%

Use the **border-radius** property to add rounded corners to a button:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
background-color: #4CAF50; /* Green */
border: none;
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
margin: 4px 2px;
cursor: pointer;
}
```

```
.button1 {border-radius: 2px;}
.button2 {border-radius: 4px;}
.button3 {border-radius: 8px;}
.button4 {border-radius: 12px;}
.button5 {border-radius: 50%;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Rounded Buttons</h2>
```


<p>Add rounded corners to a button with the border-radius property:</p>

```
<button class="button button1">2px</button>
```

```
<button class="button button2">4px</button>
```

```
<button class="button button3">8px</button>
```

```
<button class="button button4">12px</button>
```

```
<button class="button button5">50%</button>
```

```
</body>
```

```
</html>
```

Colored Button Borders

Green Blue Red Gray Black

Use the **border** property to add a colored border to a button:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
    color: white;
```

```
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
margin: 4px 2px;
cursor: pointer;
}
```

```
.button1 {
    background-color: white;
    color: black;
    border: 2px solid #4CAF50;
}
```

```
.button2 {
    background-color: white;
    color: black;
    border: 2px solid #008CBA;
}
```

```
.button3 {
```

```
background-color: white;

color: black;

border: 2px solid #f44336;
}
```

```
.button4 {

background-color: white;

color: black;

border: 2px solid #e7e7e7;
}
```

```
.button5 {

background-color: white;

color: black;

border: 2px solid #555555;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Colored Button Borders</h2>
```

```
<p>Use the border property to add a border to the button:</p>
```

```
<button class="button button1">Green</button>
```

```
<button class="button button2">Blue</button>
```

```
<button class="button button3">Red</button>
```

```
<button class="button button4">Gray</button>
```

```
<button class="button button5">Black</button>
```

```
</body>
```

```
</html>
```

Hoverable Buttons

Green Blue Red Grey Black

Green Blue Red Grey Black

Use the `:hover` selector to change the style of a button when you move the mouse over it.

Tip: Use the `transition-duration` property to determine the speed of the "hover" effect:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
border: none;

color: white;

padding: 16px 32px;

text-align: center;

text-decoration: none;

display: inline-block;

font-size: 16px;

margin: 4px 2px;

-webkit-transition-duration: 0.4s; /* Safari */

transition-duration: 0.4s;

cursor: pointer;

}
```

```
.button1 {

    background-color: white;

    color: black;

    border: 2px solid #4CAF50;

}
```

```
.button1:hover {

    background-color: #4CAF50;

    color: white;
```

```
}
```

```
.button2 {
```

```
    background-color: white;
```

```
    color: black;
```

```
    border: 2px solid #008CBA;
```

```
}
```

```
.button2:hover {
```

```
    background-color: #008CBA;
```

```
    color: white;
```

```
}
```

```
.button3 {
```

```
    background-color: white;
```

```
    color: black;
```

```
    border: 2px solid #f44336;
```

```
}
```

```
.button3:hover {
```

```
    background-color: #f44336;
```

```
    color: white;
```

```
}
```

```
.button4 {
```

```
    background-color: white;
```

```
    color: black;
```

```
    border: 2px solid #e7e7e7;
```

```
}
```

```
.button4:hover {background-color: #e7e7e7;}
```

```
.button5 {
```

```
    background-color: white;
```

```
    color: black;
```

```
    border: 2px solid #555555;
```

```
}
```

```
.button5:hover {
```

```
    background-color: #555555;
```

```
    color: white;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Hoverable Buttons</h2>
```

```
<p>Use the :hover selector to change the style of the button when you move the mouse over it.</p>
```

```
<p><strong>Tip:</strong> Use the transition-duration property to determine the speed of the "hover" effect:</p>
```

```
<button class="button button1">Green</button>
```

```
<button class="button button2">Blue</button>
```

```
<button class="button button3">Red</button>
```

```
<button class="button button4">Gray</button>
```

```
<button class="button button5">Black</button>
```

```
</body>
```

```
</html>
```

Shadow Buttons

Shadow Button Shadow on hover

Use the `box-shadow` property to add shadows to a button:

Example

```
<!DOCTYPE html>
```

```
<html>
```



```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
    color: white;
```

```
    padding: 15px 32px;
```

```
    text-align: center;
```

```
    text-decoration: none;
```

```
    display: inline-block;
```

```
    font-size: 16px;
```

```
    margin: 4px 2px;
```

```
    cursor: pointer;
```

```
    -webkit-transition-duration: 0.4s; /* Safari */
```

```
    transition-duration: 0.4s;
```

```
}
```

```
.button1 {
```

```
    box-shadow: 0 8px 16px 0 rgba(0,0,0,0.2), 0 6px 20px 0 rgba(0,0,0,0.19);
```

```
}
```

```
.button2:hover {
```

```
    box-shadow: 0 12px 16px 0 rgba(0,0,0,0.24),0 17px 50px 0 rgba(0,0,0,0.19);
}

</style>

</head>

<body>

<h2>Shadow Buttons</h2>

<p>Use the box-shadow property to add shadows to the button:</p>

<button class="button button1">Shadow Button</button>

<button class="button button2">Shadow on Hover</button>

</body>

</html>
```

Disabled Buttons

Normal Button Disabled Button

Use the **opacity** property to add transparency to a button (creates a "disabled" look).

Tip: You can also add the **cursor** property with a value of "not-allowed", which will display a "no parking sign" when you mouse over the button:

Example

```
<!DOCTYPE html>

<html>
```

```
<head>

<style>

.button {

    background-color: #4CAF50; /* Green */

    border: none;

    color: white;

    padding: 15px 32px;

    text-align: center;

    text-decoration: none;

    display: inline-block;

    font-size: 16px;

    margin: 4px 2px;

    cursor: pointer;

}

.disabled {

    opacity: 0.6;

    cursor: not-allowed;

}

</style>

</head>

<body>
```

```
<h2>Disabled Buttons</h2>
```

```
<p>Use the opacity property to add some transparency to the button (make it look disabled):</p>
```

```
<button class="button">Normal Button</button>
```

```
<button class="button disabled">Disabled Button</button>
```

```
</body>
```

```
</html>
```

Button Width

250px

50% 100%

By default, the size of the button is determined by its text content (as wide as its content). Use the `width` property to change the width of a button:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
color: white;
padding: 15px 32px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
margin: 4px 2px;
cursor: pointer;
}
```

```
.button1 {width: 250px;}
```

```
.button2 {width: 50%;}
```

```
.button3 {width: 100%;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Button Width</h2>
```

```
<p>Use the width property to change the width of the button:</p>
```

```
<p><strong>Tip:</strong> Use pixels if you want to set a fixed width and use
percent for responsive buttons (e.g. 50% of its parent element). Resize the
browser window to see the effect.</p>
```

```
<button class="button button1">250px</button><br>
```

```
<button class="button button2">50%</button><br>
```

```
<button class="button button3">100%</button>
```

```
</body>
```

```
</html>
```

Button Groups

ButtonButtonButtonButton

Remove margins and add `float:left` to each button to create a button group:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: none;
```

```
    color: white;
```

```
    padding: 15px 32px;
```

```
    text-align: center;
```

```
text-decoration: none;

display: inline-block;

font-size: 16px;

cursor: pointer;

float: left;

}
```

```
.button:hover {

    background-color: #3e8e41;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Button Groups</h2>
```

```
<p>Remove margins and float the buttons to create a button group:</p>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<p style="clear:both"><br>Remember to clear floats after, or else will this p  
element also float next to the buttons.</p>
```

```
</body>
```

```
</html>
```

Bordered Button Groups

ButtonButtonButtonButton

Use the **border** property to create a bordered button group:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    background-color: #4CAF50; /* Green */
```

```
    border: 1px solid green;
```

```
    color: white;
```

```
    padding: 15px 32px;
```

```
    text-align: center;
```

```
    text-decoration: none;
```

```
    display: inline-block;
```

```
    font-size: 16px;
```



```
    cursor: pointer;

    float: left;
}
```

```
.button:hover {

    background-color: #3e8e41;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Bordered Button Group</h2>
```

```
<p>Add borders to create a bordered button group:</p>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<button class="button">Button</button>
```

```
<p style="clear:both"><br>Remember to clear floats after, or else will this p
element also float next to the buttons.</p>
```

```
</body>
```

```
</html>
```

Animated Buttons

Example

Add an arrow on hover:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
  display: inline-block;
```

```
  border-radius: 4px;
```

```
  background-color: #f4511e;
```

```
  border: none;
```

```
  color: #FFFFFF;
```

```
  text-align: center;
```

```
  font-size: 28px;
```

```
  padding: 20px;
```

```
  width: 200px;
```

```
  transition: all 0.5s;
```

```
  cursor: pointer;
```

```
  margin: 5px;
```

```
}
```

```
.button span {  
  cursor: pointer;  
  display: inline-block;  
  position: relative;  
  transition: 0.5s;  
}
```

```
.button span:after {  
  content: '»';  
  position: absolute;  
  opacity: 0;  
  top: 0;  
  right: -20px;  
  transition: 0.5s;  
}
```

```
.button:hover span {  
  padding-right: 25px;  
}
```

```
.button:hover span:after {  
  opacity: 1;  
  right: 0;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Animated Button</h2>
```

```
<button class="button" style="vertical-align:middle"><span>Hover  
</span></button>
```

```
</body>
```

```
</html>
```

Example

Add a "ripple" effect on click:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {  
    position: relative;  
    background-color: #4CAF50;  
    border: none;  
    font-size: 28px;  
    color: #FFFFFF;  
    padding: 20px;  
    width: 200px;  
    text-align: center;  
    -webkit-transition-duration: 0.4s; /* Safari */  
    transition-duration: 0.4s;  
    text-decoration: none;  
    overflow: hidden;  
    cursor: pointer;  
}
```

```
.button:after {  
    content: "";  
    background: #f1f1f1;  
    display: block;  
    position: absolute;
```

```
padding-top: 300%;  
padding-left: 350%;  
margin-left: -20px !important;  
margin-top: -120%;  
opacity: 0;  
transition: all 0.8s  
}
```

```
.button:active:after {  
padding: 0;  
margin: 0;  
opacity: 1;  
transition: 0s  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Animated Button - Ripple Effect</h2>
```

```
<button class="button">Click Me</button>
```

```
</body>
```

```
</html>
```

Example

Add a "pressed" effect on click:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.button {
```

```
    display: inline-block;
```

```
    padding: 15px 25px;
```

```
    font-size: 24px;
```

```
    cursor: pointer;
```

```
    text-align: center;
```

```
    text-decoration: none;
```

```
    outline: none;
```

```
    color: #fff;
```

```
    background-color: #4CAF50;
```

```
    border: none;
```

```
    border-radius: 15px;
```

```
    box-shadow: 0 9px #999;
```

```
}
```

```
.button:hover {background-color: #3e8e41}
```

```
.button:active {  
  background-color: #3e8e41;  
  box-shadow: 0 5px #666;  
  transform: translateY(4px);  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Animated Buttons - "Pressed Effect"</h2>
```

```
<button class="button">Click Me</button>
```

```
</body>
```

```
</html>
```

CSS Pagination Examples

[< Previous](#) [Next >](#)

Learn how to create a responsive pagination using CSS.

Simple Pagination

If you have a website with lots of pages, you may wish to add some sort of pagination to each page:

- «
- 1
- 2
- 3
- 4
- 5
- 6
- »
- ⏪
- ⏩

To create a pagination, style a HTML list:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Simple Pagination</h2>
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>
```

```
<li><a href="#">4</a></li>
```

```
<li><a href="#">5</a></li>
```

```
<li><a href="#">6</a></li>
```

```
<li><a href="#">7</a></li>
```

```
<li><a href="#">»</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Rounded Active and Hoverable Buttons

- [«](#)
- [1](#)
- [2](#)
- [3](#)
- [4](#)
- [5](#)
- [6](#)
- [7](#)
- [»](#)

Add the **border-radius** property if you want a rounded "active" and "hover" button:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    border-radius: 5px;  
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
    border-radius: 5px;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Rounded Active and Hover Buttons</h2>
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>
```

```
<li><a href="#">4</a></li>
```

```
<li><a href="#">5</a></li>
```

```
<li><a href="#">6</a></li>
```

```
<li><a href="#">7</a></li>
```

```
<li><a href="#">»</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Hoverable Transition Effect

- «
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- »

Add the **transition** property to the page links to create a transition effect on hover:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {
```

```
    color: black;
```

```
    float: left;
```

```
    padding: 8px 16px;
```

```
    text-decoration: none;
```

```
    transition: background-color .3s;
```

```
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Transition Effect on Hover</h2>
```

```
<p>Move the mouse over the numbers.</p>
```

```
<ul class="pagination">
```

```
  <li><a href="#">«</a></li>
```

```
  <li><a href="#">1</a></li>
```

```
  <li><a class="active" href="#">2</a></li>
```

```
  <li><a href="#">3</a></li>
```

```
  <li><a href="#">4</a></li>
```

```
  <li><a href="#">5</a></li>
```

```
  <li><a href="#">6</a></li>
```

```
  <li><a href="#">7</a></li>
```

```
<li><a href="#">»</a></li>

</ul>
```

```
</body>
```

```
</html>
```

Bordered Pagination

- <<
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- >>

Use the **border** property to add borders to the pagination:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```



```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    transition: background-color .3s;  
    border: 1px solid #ddd;  
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
    border: 1px solid #4CAF50;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Bordered Pagination</h2>

<ul class="pagination">

  <li><a href="#">«</a></li>

  <li><a href="#">1</a></li>

  <li><a class="active" href="#">2</a></li>

  <li><a href="#">3</a></li>

  <li><a href="#">4</a></li>

  <li><a href="#">5</a></li>

  <li><a href="#">6</a></li>

  <li><a href="#">7</a></li>

  <li><a href="#">»</a></li>

</ul>
```

```
</body>
```

```
</html>
```

Rounded Borders

Tip: Add rounded borders to your first and last link in the pagination:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {  
    display: inline-block;  
    padding: 0;  
    margin: 0;  
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    transition: background-color .3s;  
    border: 1px solid #ddd;  
}
```

```
.pagination li:first-child a {  
    border-top-left-radius: 5px;  
    border-bottom-left-radius: 5px;  
}
```

```
.pagination li:last-child a {  
    border-top-right-radius: 5px;  
    border-bottom-right-radius: 5px;  
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
    border: 1px solid #4CAF50;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Pagination with Rounded Borders</h2>
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>

<li><a href="#">4</a></li>

<li><a href="#">5</a></li>

<li><a href="#">6</a></li>

<li><a href="#">7</a></li>

<li><a href="#">»</a></li>

</ul>

</body>

</html>
```

Space Between Links

Tip: Add the `margin` property if you do not want to group the page links:

- «
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- »

Example

```
<!DOCTYPE html>

<html>

<head>
```

```
<style>
```

```
ul.pagination {  
    display: inline-block;  
    padding: 0;  
    margin: 0;  
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    transition: background-color .3s;  
    border: 1px solid #ddd;  
    margin: 0 4px;  
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;
```

```
border: 1px solid #4CAF50;
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Pagination with Margins</h2>
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>
```

```
<li><a href="#">4</a></li>
```

```
<li><a href="#">5</a></li>
```

```
<li><a href="#">6</a></li>
```

```
<li><a href="#">7</a></li>
```

```
<li><a href="#">»</a></li>
```

```
</ul>
```

```
</body>
```

</html>

Pagination Size

- <<
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- >>

Change the size of the pagination with the **font-size** property:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```

```
ul.pagination li {display: inline;}
```



```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    transition: background-color .3s;  
    border: 1px solid #ddd;  
    font-size: 22px;  
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
    border: 1px solid #4CAF50;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}  
  
</style>  
  
</head>  
  
<body>
```

```
<h2>Pagination Size</h2>
```

```
<p>Change the font-size property to make the pagination smaller or bigger.</p>
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>
```

```
<li><a href="#">4</a></li>
```

```
<li><a href="#">5</a></li>
```

```
<li><a href="#">6</a></li>
```

```
<li><a href="#">7</a></li>
```

```
<li><a href="#">»</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Centered Pagination

- «
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- »

To center the pagination, wrap a container element around it (like <div>), and add `text-align:center`

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.pagination {
```

```
    display: inline-block;
```

```
    padding: 0;
```

```
    margin: 0;
```

```
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {
```

```
    color: black;
```

```
    float: left;
```

```
    padding: 8px 16px;
```

```
    text-decoration: none;
```

```
    transition: background-color .3s;
```

```
    border: 1px solid #ddd;
```

```
}
```

```
ul.pagination li a.active {  
    background-color: #4CAF50;  
    color: white;  
    border: 1px solid #4CAF50;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
div.center {text-align: center;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Centered Pagination</h2>
```

```
<div class="center">
```

```
<ul class="pagination">
```

```
<li><a href="#">«</a></li>
```

```
<li><a href="#">1</a></li>
```

```
<li><a class="active" href="#">2</a></li>
```

```
<li><a href="#">3</a></li>
<li><a href="#">4</a></li>
<li><a href="#">5</a></li>
<li><a href="#">6</a></li>
<li><a href="#">7</a></li>
<li><a href="#">»</a></li>
</ul>
</div>
```

```
</body>
```

```
</html>
```

More Examples

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
ul.pagination {
    display: inline-block;
    padding: 0;
    margin: 0;
}
```

```
ul.pagination li {display: inline;}
```

```
ul.pagination li a {  
    color: black;  
    float: left;  
    padding: 8px 16px;  
    text-decoration: none;  
    transition: background-color .3s;  
    border: 1px solid #ddd;  
    font-size: 18px;  
}
```

```
ul.pagination li a.active {  
    background-color: #eee;  
    color: black;  
    border: 1px solid #ddd;  
}
```

```
ul.pagination li a:hover:not(.active) {background-color: #ddd;}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p>Next/Previous buttons:</p>
```

```
<ul class="pagination">
```

```
<li><a href="#">⏪</a></li>
```

```
<li><a href="#">⏩</a></li>
```

```
</ul>
```

```
<p>Navigation pagination:</p>
```

```
<ul class="pagination">
```

```
<li><a href="#" class="active">Home</a></li>
```

```
<li><a href="#">Link 1</a></li>
```

```
<li><a href="#">Link 2</a></li>
```

```
<li><a href="#">Link 3</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Breadcrumbs

- [Home](#)

- [Pictures](#)

- [Summer 15](#)

- [Italy](#)

Another variation of pagination is so-called "breadcrumbs":

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul.breadcrumb {
```

```
    padding: 8px 16px;
```

```
    list-style: none;
```

```
    background-color: #eee;
```

```
}
```

```
ul.breadcrumb li {display: inline;}
```

```
ul.breadcrumb li+li:before {
```

```
    padding: 8px;
```

```
    color: black;
```

```
    content: "\00a0";
```

```
}
```

```
ul.breadcrumb li a {color: green;}
```



```
</style>

</head>

<body>


<h2>Breadcrumb Pagination</h2>

<ul class="breadcrumb">

  <li><a href="#">Home</a></li>

  <li><a href="#">Pictures</a></li>

  <li><a href="#">Summer 15</a></li>

  <li>Italy</li>

</ul>


</body>

</html>
```

CSS3 Multiple Columns

[⏪ Previous](#)[Next ⏩](#)

CSS3 Multi-column Layout

The CSS3 multi-column layout allows easy definition of multiple columns of text
- just like in newspapers:

Daily Ping

Lorem ipsum

dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit- or -moz- specify the first version that worked with a prefix.

Property					
column-count	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit- 11.1
column-gap	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit- 11.1
column-rule	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit- 11.1

column-rule-color	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit 11.1
column-rule-style	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit 11.1
column-rule-width	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit 11.1
column-span	50.0 4.0 -webkit-	10.0	Not supported	9.0 3.1 -webkit-	37.0 15.0 -webkit 11.1
column-width	50.0 4.0 -webkit-	10.0	2.0 -moz-	9.0 3.1 -webkit-	37.0 15.0 -webkit 11.1

CSS3 Multi-column Properties

In this chapter you will learn about the following multi-column properties:

- column-count
- column-gap
- column-rule-style
- column-rule-width
- column-rule-color
- column-rule
- column-span
- column-width

CSS3 Create Multiple Columns

The `column-count` property specifies the number of columns an element should be divided into.

The following example will divide the text in the `<div>` element into 3 columns:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.newspaper {

    -webkit-column-count: 3; /* Chrome, Safari, Opera */

    -moz-column-count: 3; /* Firefox */

    column-count: 3;

}

</style>

</head>

<body>
```

Note: Internet Explorer 9, and earlier versions, does not support the `column-count` property.

```
<div class="newspaper">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

</div>

</body>

</html>

CSS3 Specify the Gap Between Columns

The `column-gap` property specifies the gap between the columns.

The following example specifies a 40 pixels gap between the columns:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
    -moz-column-count: 3; /* Firefox */
```

```
column-count: 3;

-webkit-column-gap: 40px; /* Chrome, Safari, Opera */
-moz-column-gap: 40px; /* Firefox */
column-gap: 40px;

}

</style>

</head>

<body>
```

<p>Note: Internet Explorer 9, and earlier versions, does not support the column-gap property.</p>

```
<div class="newspaper">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

```
</div>
```

```
</body>
```

</html>

CSS3 Column Rules

The `column-rule-style` property specifies the style of the rule between columns:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
    -moz-column-count: 3; /* Firefox */
```

```
    column-count: 3;
```

```
    -webkit-column-gap: 40px; /* Chrome, Safari, Opera */
```

```
    -moz-column-gap: 40px; /* Firefox */
```

```
    column-gap: 40px;
```

```
    -webkit-column-rule-style: solid; /* Chrome, Safari, Opera */
```

```
    -moz-column-rule-style: solid; /* Firefox */
```

```
    column-rule-style: solid;
```

```
}
```

```
</style>
```

```
</head>
```

<body>

<p>Note: Internet Explorer 9, and earlier versions, does not support the column-rule-style property.</p>

<div class="newspaper">

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

</div>

</body>

</html>

The **column-rule-width** property specifies the width of the rule between columns:

Example

<!DOCTYPE html>

<html>

<head>

<style>

.newspaper {

-webkit-column-count: 3; /* Chrome, Safari, Opera */

-moz-column-count: 3; /* Firefox */

column-count: 3;

-webkit-column-gap: 40px; /* Chrome, Safari, Opera */

-moz-column-gap: 40px; /* Firefox */

column-gap: 40px;

-webkit-column-rule-style: solid; /* Chrome, Safari, Opera */

-moz-column-rule-style: solid; /* Firefox */

column-rule-style: solid;

-webkit-column-rule-width: 1px; /* Chrome, Safari, Opera */

-moz-column-rule-width: 1px; /* Firefox */

column-rule-width: 1px;

}

</style>

</head>

<body>

<p>Note: Internet Explorer 9, and earlier versions, does not support the column-rule-width property.</p>

```
<div class="newspaper">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exercitation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

```
</div>
```

```
</body>
```

```
</html>
```

The `column-rule-color` property specifies the color of the rule between columns:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
    -moz-column-count: 3; /* Firefox */
```

```
    column-count: 3;
```

```
-webkit-column-gap: 40px; /* Chrome, Safari, Opera */
-moz-column-gap: 40px; /* Firefox */
column-gap: 40px;

-webkit-column-rule-style: solid; /* Chrome, Safari, Opera */
-moz-column-rule-style: solid; /* Firefox */
column-rule-style: solid;

-webkit-column-rule-width: 1px; /* Chrome, Safari, Opera */
-moz-column-rule-width: 1px; /* Firefox */
column-rule-width: 1px;

-webkit-column-rule-color: lightblue; /* Chrome, Safari, Opera */
-moz-column-rule-color: lightblue; /* Firefox */
column-rule-color: lightblue;
}

</style>

</head>

<body>
```

<p>Note: Internet Explorer 9, and earlier versions, does not support the column-rule-color property.</p>

<div class="newspaper">

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

</div>

</body>

</html>

The **column-rule** property is a shorthand property for setting all the column-rule-* properties above.

The following example sets the width, style, and color of the rule between columns:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
  -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
  -moz-column-count: 3; /* Firefox */
```

```
column-count: 3;

-webkit-column-gap: 40px; /* Chrome, Safari, Opera */
-moz-column-gap: 40px; /* Firefox */
column-gap: 40px;

-webkit-column-rule: 1px solid lightblue; /* Chrome, Safari, Opera */
-moz-column-rule: 1px solid lightblue; /* Firefox */
column-rule: 1px solid lightblue;
}

</style>

</head>

<body>
```

<p>Note: Internet Explorer 9, and earlier versions, does not support the column-rule property.</p>

<div class="newspaper">

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

```
</div>
```

```
</body>
```

```
</html>
```

Specify How Many Columns an Element Should Span

The `column-span` property specifies how many columns an element should span across.

The following example specifies that the `<h2>` element should span across all columns:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
    -moz-column-count: 3; /* Firefox */
```

```
    column-count: 3;
```

```
    -webkit-column-gap: 40px; /* Chrome, Safari, Opera */
```

```
    -moz-column-gap: 40px; /* Firefox */
```

```
    column-gap: 40px;
```

```
    -webkit-column-rule: 1px solid lightblue; /* Chrome, Safari, Opera */
```

```
-moz-column-rule: 1px solid lightblue; /* Firefox */  
column-rule: 1px solid lightblue;  
}  
  
h2 {  
    -webkit-column-span: all; /* Chrome, Safari, Opera */  
    column-span: all;  
}  
  
</style>  
</head>  
<body>
```

<p>Note: Firefox and Internet Explorer 9 (and earlier versions) do not support the column-span property.</p>

<div class="newspaper">

<h2>Lorem Ipsum Dolor Sit Amet</h2>

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum

soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

</div>

</body>

</html>

Specify The Column Width

The `column-width` property specifies a suggested, optimal width for the columns.

The following example specifies that the suggested, optimal width for the columns should be 100px:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.newspaper {
```

```
    -webkit-column-count: 3; /* Chrome, Safari, Opera */
```

```
    -moz-column-count: 3; /* Firefox */
```

```
    column-count: 3;
```

```
    -webkit-column-width: 100px; /* Chrome, Safari, Opera */
```

```
    -moz-column-width: 100px; /* Firefox */
```

```
    column-width: 100px;
```



```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><b>Note:</b> Internet Explorer 9, and earlier versions, does not support the  
column-width property.</p>
```

```
<div class="newspaper">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

```
</div>
```

```
</body>
```

```
</html>
```

CSS3 Multi-columns Properties

The following table lists all the multi-columns properties:

Property	Description
column-count	Specifies the number of columns an element should be divided into
column-fill	Specifies how to fill columns
column-gap	Specifies the gap between the columns
column-rule	A shorthand property for setting all the column-rule-* properties
column-rule-color	Specifies the color of the rule between columns
column-rule-style	Specifies the style of the rule between columns
column-rule-width	Specifies the width of the rule between columns
column-span	Specifies how many columns an element should span across
column-width	Specifies a suggested, optimal width for the columns

CSS3 User Interface

[↩ Previous](#) [Next ↪](#)

CSS3 User Interface

CSS3 has new user interface features such as resizing elements, outlines, and box sizing.

In this chapter you will learn about the following user interface properties:

- `resize`
- `outline-offset`

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by `-webkit-` or `-moz-` specify the first version that worked with a prefix.

Property					
resize	4.0	Not supported	5.0 4.0 -moz-	4.0	15.0
outline-offset	4.0	Not supported	5.0 4.0 -moz-	4.0	9.5

CSS3 Resizing

The `resize` property specifies whether or not an element should be resizable by the user.

This div element is resizable by the user (works in Chrome, Firefox, Safari and Opera).

The following example lets the user resize only the width of a `<div>` element:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

div {

    border: 2px solid;

    padding: 20px;

    width: 300px;

    resize: horizontal;

    overflow: auto;

}

</style>

</head>

<body>
```

<p>Note: Internet Explorer does not support the resize property.</p>

<div>Let the user resize the width of this div element.</div>

</body>

</html>

The following example lets the user resize only the height of a <div> element:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    border: 2px solid;
```

```
    padding: 20px;
```

```
    width: 300px;
```

```
    resize: vertical;
```

```
    overflow: auto;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><b>Note:</b> Internet Explorer does not support the resize property.</p>
```

```
<div>Let the user resize the height of this div element.</div>
```

```
</body>
```

```
</html>
```

The following example lets the user resize both the height and the width of a `<div>` element:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
    border: 2px solid;
```

```
    padding: 20px;
```

```
    width: 300px;
```

```
    resize: both;
```

```
    overflow: auto;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><b>Note:</b> Internet Explorer does not support the resize property.</p>
```

```
<div>Let the user resize both the height and the width of this div element.</div>
```

```
</body>
```

```
</html>
```

CSS3 Outline Offset

The **outline-offset** property adds space between an outline and the edge or border of an element.

Outlines differ from borders in three ways:

- An outline is a line drawn around elements, outside the border edge
- An outline does not take up space
- An outline may be non-rectangular

This div has an outline 15px outside the border edge.

The following example uses the outline-offset property to add a 15px space between the border and the outline:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
div {
```

```
margin: 20px;

padding: 10px;

width: 300px;

height: 100px;

border: 1px solid black;

outline: 1px solid red;

/* Move the outline 15px away from the border */

outline-offset: 15px;
}

</style>

</head>

<body>

<p><b>Note:</b> Internet Explorer does not support the outline-offset
property.</p>

<div>This div has an outline 15px outside the border edge.</div>

</body>

</html>
```

CSS3 User Interface Properties

The following table lists all the user interface properties:

Property	Description
box-sizing	Allows you to include the padding and border in an element's total width and height
nav-down	Specifies where to navigate when using the arrow-down navigation key
nav-index	Specifies the tabbing order for an element
nav-left	Specifies where to navigate when using the arrow-left navigation key
nav-right	Specifies where to navigate when using the arrow-right navigation key
nav-up	Specifies where to navigate when using the arrow-up navigation key
outline-offset	Adds space between an outline and the edge or border of an element
resize	Specifies whether or not an element is resizable by the user

CSS3 Box Sizing

[< Previous](#)
[Next >](#)

CSS3 Box Sizing

The CSS3 **box-sizing** property allows us to include the padding and border in an element's total width and height.

Browser Support

The numbers in the table specify the first browser version that fully supports the property.

Numbers followed by -webkit- or -moz- specify the first version that worked with a prefix.

Property					
box-sizing	10.0 4.0 -webkit-	8.0	29.0 2.0 -moz-	5.1 3.1 -webkit-	9.5

Without the CSS3 box-sizing Property

By default, the width and height of an element is calculated like this:

width + padding + border = actual width of an element
height + padding + border = actual height of an element

This means: When you set the width/height of an element, the element often appear bigger than you have set (because the element's border and padding are added to the element's specified width/height).

The following illustration shows two <div> elements with the same specified width and height:

This div is smaller (width is 300px and height is 100px).

This div is bigger (width is also 300px and height is 100px).

The two <div> elements above end up with different sizes in the result (because div2 has a padding specified):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.div1 {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    border: 1px solid blue;
```

```
}
```

```
.div2 {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 50px;
```

```
    border: 1px solid red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="div1">This div is smaller (width is 300px and height is 100px).</div>
```

```
<br>
```

```
<div class="div2">This div is bigger (width is also 300px and height is  
100px).</div>
```

```
</body>
```

```
</html>
```

So, for a long time web developers have specified a smaller width value than they wanted, because they had to subtract out the padding and borders.

With CSS3, the `box-sizing` property solves this problem.

With the CSS3 box-sizing Property

The CSS3 `box-sizing` property allows us to include the padding and border in an element's total width and height.

If you set `box-sizing: border-box;` on an element padding and border are included in the width and height:

Both divs are the same size now!

Hooray!

Here is the same example as above, with `box-sizing: border-box;` added to both `<div>` elements:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.div1 {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    border: 1px solid blue;
```

```
    box-sizing: border-box;
```

```
}
```

```
.div2 {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 50px;
```

```
    border: 1px solid red;
```

```
    box-sizing: border-box;
```

```
}
```

```
</style>

</head>

<body>

<div class="div1">Both divs are the same size now!</div>

<br>

<div class="div2">Hooray!</div>


</body>

</html>
```

Here is the same example as above, with `box-sizing: border-box;` added to both `<div>` elements:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.div1 {

    width: 300px;

    height: 100px;

    border: 1px solid blue;

    box-sizing: border-box;
```

```
}
```

```
.div2 {
```

```
    width: 300px;
```

```
    height: 100px;
```

```
    padding: 50px;
```

```
    border: 1px solid red;
```

```
    box-sizing: border-box;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="div1">Both divs are the same size now!</div>
```

```
<br>
```

```
<div class="div2">Hooray!</div>
```

```
</body>
```

```
</html>
```

Since the result of using the `box-sizing: border-box;` is so much better, many developers want all elements on their pages to work this way.

The code below ensures that all elements are sized in this more intuitive way. Many browsers already use `box-sizing: border-box;` for many form elements (but not all - which is why inputs and text areas look different at width: 100%;).

Applying this to all elements is safe and wise:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

* {

    box-sizing: border-box;

}

input, textarea {

    width: 100%;

}

</style>

</head>

<body>

<form action="action_page.php">

First name:<br>

<input type="text" name="firstname" value="Mickey"><br>
```


Last name:

<input type="text" name="lastname" value="Mouse">

Comments:

<textarea name="message" rows="5" cols="30">

</textarea>

<input type="submit" value="Submit">

</form>

<p>Tip: Try to remove the box-sizing property from the style element and look what happens.

Notice that the input, textarea, and submit button will no longer have equal widths.</p>

</body>

</html>

CSS3 Flexible Box

[< Previous](#)[Next >](#)

CSS3 Flexbox

Flexible boxes, or flexbox, is a new layout mode in CSS3.

Use of flexbox ensures that elements behave predictably when the page layout must accommodate different screen sizes and different display devices.

For many applications, the flexible box model provides an improvement over the block model in that it does not use floats, nor do the flex container's margins collapse with the margins of its contents.

Browser Support

The numbers in the table specify the first browser version that fully supports the feature.

Numbers followed by -webkit- or -moz- specify the first version that worked with a prefix.

Property					
Basic support (single-line flexbox)	29.0 21.0 -webkit-	11.0	22.0 18.0 -moz-	6.1 -webkit-	12.1 -webkit-
Multi-line flexbox	29.0 21.0 -webkit-	11.0	28.0	6.1 -webkit-	17.0 15.0 -webkit- 12.1

CSS3 Flexbox Concepts

Flexbox consists of flex containers and flex items.

A flex container is declared by setting the `display` property of an element to either `flex` (rendered as a block) or `inline-flex` (rendered as inline).

Inside a flex container there is one or more flex items.

Note: Everything outside a flex container and inside a flex item is rendered as usual. Flexbox defines how flex items are laid out inside a flex container.

Flex items are positioned inside a flex container along a flex line. By default there is only one flex line per flex container.

The following example shows three flex items. They are positioned by default: along the horizontal flex line, from left to right:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
    display: -webkit-flex;
```

```
    display: flex;
```

```
    width: 400px;
```

```
    height: 250px;
```

```
    background-color: lightgrey;
```

```
}
```

```
.flex-item {
```

```
    background-color: cornflowerblue;
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    margin: 10px;
```

```
}
```

```
</style>
```

```
</head>

<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>

</html>
```

It is also possible to change the direction of the flex line.

If we set the **direction** property to **rtl** (right-to-left), the text is drawn right to left, and also the flex line changes direction, which will change the page layout:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

body {

  direction: rtl;

}
```

```
.flex-container {  
    display: -webkit-flex;  
    display: flex;  
    width: 400px;  
    height: 250px;  
    background-color: lightgrey;  
}
```

```
.flex-item {  
    background-color: cornflowerblue;  
    width: 100px;  
    height: 100px;  
    margin: 10px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
  <div class="flex-item">flex item 1</div>
```

```
  <div class="flex-item">flex item 2</div>
```

```
<div class="flex-item">flex item 3</div>
</div>
```

```
</body>
```

```
</html>
```

Flex Direction

The `flex-direction` property specifies the direction of the flexible items inside the flex container. The default value of `flex-direction` is `row` (left-to-right, top-to-bottom).

The other values are as follows:

- `row-reverse` - If the writing-mode (direction) is left to right, the flex items will be laid out right to left
- `column` - If the writing system is horizontal, the flex items will be laid out vertically
- `column-reverse` - Same as column, but reversed

The following example shows the result of using the `row-reverse` value:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.flex-container {

    display: -webkit-flex;

    display: flex;
```

```
-webkit-flex-direction: row-reverse;

flex-direction: row-reverse;

width: 400px;

height: 250px;

background-color: lightgrey;

}
```

```
.flex-item {

    background-color: cornflowerblue;

    width: 100px;

    height: 100px;

    margin: 10px;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
    <div class="flex-item">flex item 1</div>
```

```
    <div class="flex-item">flex item 2</div>
```

```
    <div class="flex-item">flex item 3</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

The following example shows the result of using the `column` value:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
    display: -webkit-flex;
```

```
    display: flex;
```

```
    -webkit-flex-direction: column;
```

```
    flex-direction: column;
```

```
    width: 400px;
```

```
    height: 250px;
```

```
    background-color: lightgrey;
```

```
}
```

```
.flex-item {
```

```
    background-color: cornflowerblue;
```

```
    width: 100px;
```



```
    height: 100px;
    margin: 10px;
}
</style>
</head>
<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>
</html>
```

The following example shows the result of using the `column-reverse` value:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.flex-container {
```

```
display: -webkit-flex;

display: flex;

-webkit-flex-direction: column-reverse;

flex-direction: column-reverse;

width: 400px;

height: 250px;

background-color: lightgrey;

}
```

```
.flex-item {

    background-color: cornflowerblue;

    width: 100px;

    height: 100px;

    margin: 10px;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
  <div class="flex-item">flex item 1</div>
```

```
  <div class="flex-item">flex item 2</div>
```

```
<div class="flex-item">flex item 3</div>

</div>
```

```
</body>
```

```
</html>
```

The justify-content Property

The **justify-content** property horizontally aligns the flexible container's items when the items do not use all available space on the main-axis.

The possible values are as follows:

- **flex-start** - Default value. Items are positioned at the beginning of the container
- **flex-end** - Items are positioned at the end of the container
- **center** - Items are positioned at the center of the container
- **space-between** - Items are positioned with space between the lines
- **space-around** - Items are positioned with space before, between, and after the lines

The following example shows the result of using the **flex-end** value:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
  display: -webkit-flex;
```

```
  display: flex;
```

```
-webkit-justify-content: flex-end;  
justify-content: flex-end;  
  
width: 400px;  
  
height: 250px;  
  
background-color: lightgrey;  
  
}
```

```
.flex-item {  
  
    background-color: cornflowerblue;  
  
    width: 100px;  
  
    height: 100px;  
  
    margin: 10px;  
  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
    <div class="flex-item">flex item 1</div>
```

```
    <div class="flex-item">flex item 2</div>
```

```
    <div class="flex-item">flex item 3</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

The following example shows the result of using the `center` value:

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
    display: -webkit-flex;
```

```
    display: flex;
```

```
    -webkit-justify-content: center;
```

```
    justify-content: center;
```

```
    width: 400px;
```

```
    height: 250px;
```

```
    background-color: lightgrey;
```

```
}
```

```
.flex-item {
```

```
    background-color: cornflowerblue;
```

```
    width: 100px;
```

```
    height: 100px;
    margin: 10px;
}
</style>
</head>
<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>
</html>
```

The following example shows the result of using the `space-between` value:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.flex-container {
```

```
display: -webkit-flex;

display: flex;

-webkit-justify-content: space-between;

justify-content: space-between;

width: 400px;

height: 250px;

background-color: lightgrey;

}
```

```
.flex-item {

    background-color: cornflowerblue;

    width: 100px;

    height: 100px;

    margin: 10px;

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
  <div class="flex-item">flex item 1</div>
```

```
  <div class="flex-item">flex item 2</div>
```

```
<div class="flex-item">flex item 3</div>
</div>

</body>
</html>
```

The following example shows the result of using the `space-around` value:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
    display: -webkit-flex;
    display: flex;
    -webkit-justify-content: space-around;
    justify-content: space-around;
    width: 400px;
    height: 250px;
    background-color: lightgrey;
}

.flex-item {
```



```
background-color: cornflowerblue;

width: 100px;

height: 100px;

margin: 10px;
}

</style>

</head>

<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>

</html>
```

The align-items Property

The **align-items** property vertically aligns the flexible container's items when the items do not use all available space on the cross-axis.

The possible values are as follows:

- **stretch** - Default value. Items are stretched to fit the container

- **flex-start** - Items are positioned at the top of the container
- **flex-end** - Items are positioned at the bottom of the container
- **center** - Items are positioned at the center of the container (vertically)
- **baseline** - Items are positioned at the baseline of the container

The following example shows the result of using the **stretch** value (this is the default value):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
    display: -webkit-flex;
```

```
    display: flex;
```

```
    -webkit-align-items: stretch;
```

```
    align-items: stretch;
```

```
    width: 400px;
```

```
    height: 250px;
```

```
    background-color: lightgrey;
```

```
}
```

```
.flex-item {
```

```
    background-color: cornflowerblue;
```

```
    width: 100px;
```

```
margin: 10px;
}
</style>
</head>
<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>
</html>
```

The flex-wrap Property

The `flex-wrap` property specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.

The possible values are as follows:

- `nowrap` - Default value. The flexible items will not wrap
- `wrap` - The flexible items will wrap if necessary
- `wrap-reverse` - The flexible items will wrap, if necessary, in reverse order

The following example shows the result of using the `nowrap` value (this is the default value):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
    display: -webkit-flex;
```

```
    display: flex;
```

```
    -webkit-flex-wrap: nowrap;
```

```
    flex-wrap: nowrap;
```

```
    width: 300px;
```

```
    height: 250px;
```

```
    background-color: lightgrey;
```

```
}
```

```
.flex-item {
```

```
    background-color: cornflowerblue;
```

```
    width: 100px;
```

```
    height: 100px;
```

```
    margin: 10px;
```

```
}
```

```
</style>
```

```
</head>

<body>

<div class="flex-container">

  <div class="flex-item">flex item 1</div>

  <div class="flex-item">flex item 2</div>

  <div class="flex-item">flex item 3</div>

</div>

</body>

</html>
```

The following example shows the result of using the `wrap` value:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.flex-container {

  display: -webkit-flex;

  display: flex;

  -webkit-flex-wrap: wrap;

  flex-wrap: wrap;
```

```
width: 300px;

height: 250px;

background-color: lightgrey;
}
```

```
.flex-item {

    background-color: cornflowerblue;

    width: 100px;

    height: 100px;

    margin: 10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="flex-container">
```

```
    <div class="flex-item">flex item 1</div>
```

```
    <div class="flex-item">flex item 2</div>
```

```
    <div class="flex-item">flex item 3</div>
```

```
</div>
```

```
</body>
```

</html>

CSS3 Media Queries

[< Previous](#) [Next >](#)

CSS2 Introduced Media Types

The `@media` rule, introduced in CSS2, made it possible to define different style rules for different media types.

Examples: You could have one set of style rules for computer screens, one for printers, one for handheld devices, one for television-type devices, and so on.

Unfortunately these media types never got a lot of support by devices, other than the print media type.

CSS3 Introduces Media Queries

Media queries in CSS3 extend the CSS2 media types idea: Instead of looking for a type of device, they look at the capability of the device.

Media queries can be used to check many things, such as:

- width and height of the viewport
- width and height of the device
- orientation (is the tablet/phone in landscape or portrait mode?)
- resolution

Using media queries are a popular technique for delivering a tailored style sheet to tablets, iPhone, and Androids.

Browser Support

The numbers in the table specifies the first browser version that fully supports the `@media` rule.

Property					
@media	21.0	9.0	3.5	4.0	9.0

Media Query Syntax

A media query consists of a media type and can contain one or more expressions, which resolve to either true or false.

```
@media not|only mediatype and (expressions) {
  CSS-Code;
}
```

The result of the query is true if the specified media type matches the type of device the document is being displayed on and all expressions in the media query are true. When a media query is true, the corresponding style sheet or style rules are applied, following the normal cascading rules.

Unless you use the not or only operators, the media type is optional and the **all** type will be implied.

You can also have different stylesheets for different media:

```
<link rel="stylesheet" media="mediatype and|not|only (expressions)" href="print.css">
```

CSS3 Media Types

Value	Description

all	Used for all media type devices
print	Used for printers
screen	Used for computer screens, tablets, smart-phones etc.
speech	Used for screenreaders that "reads" the page out loud

Media Queries Simple Examples

One way to use media queries is to have an alternate CSS section right inside your style sheet.

The following example changes the background-color to lightgreen if the viewport is 480 pixels wide or wider (if the viewport is less than 480 pixels, the background-color will be pink):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body {  
    background-color: pink;  
}
```

```
@media screen and (min-width: 480px) {  
    body {  
        background-color: lightgreen;  
    }  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Resize the browser window to see the effect!</h1>
```

```
<p>The media query will only apply if the media type is screen and the viewport is  
480px wide or wider.</p>
```

```
</body>
```

```
</html>
```

The following example shows a menu that will float to the left of the page if the viewport is 480 pixels wide or wider (if the viewport is less than 480 pixels, the menu will be on top of the content):

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<style>
```

```
.wrapper {overflow: auto;}
```

```
#main {margin-left: 4px;}
```

```
#leftsidebar {  
    float: none;  
    width: auto;  
}
```

```
#menulist {  
    margin: 0;  
    padding: 0;  
}
```

```
.menuitem {  
    background: #CDF0F6;
```

```
border: 1px solid #d4d4d4;

border-radius: 4px;

list-style-type: none;

margin: 4px;

padding: 2px;
}

@media screen and (min-width: 480px) {

  #leftsidebar {width: 200px; float: left;}

  #main {margin-left: 216px;}
}

</style>

</head>

<body>
```

```
<div class="wrapper">

  <div id="leftsidebar">

    <ul id="menulist">

      <li class="menuitem">Menu-item 1</li>

      <li class="menuitem">Menu-item 2</li>

      <li class="menuitem">Menu-item 3</li>

      <li class="menuitem">Menu-item 4</li>
```

```
<li class="menuitem">Menu-item 5</li>

</ul>

</div>

<div id="main">

  <h1>Resize the browser window to see the effect!</h1>

  <p>This example shows a menu that will float to the left of the page if the
viewport is 480 pixels wide or wider. If the viewport is less than 480 pixels, the
menu will be on top of the content.</p>

</div>

</div>

</body>

</html>
```

CSS3 Media Queries - Examples

[⏪ Previous](#)[Next ⏩](#)

CSS3 Media Queries - More Examples

Let us look at some more examples of using media queries.

We will start with a list of names which function as email links. The HTML is:

Example 1

```
<!DOCTYPE html>

<html>
```

```
<head>

<style>

ul {

    list-style-type: none;

}

ul li a {

    color: green;

    text-decoration: none;

    padding: 3px;

    display: block;

}

</style>

</head>

<body>

<ul>

    <li><a data-email="johndoe@example.com"
href="mailto:johndoe@example.com">John Doe</a></li>

    <li><a data-email="marymoe@example.com"
href="mailto:marymoe@example.com">Mary Moe</a></li>

    <li><a data-email="amandapanda@example.com"
href="mailto:amandapanda@example.com">Amanda Panda</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Notice the `data-email` attribute. In HTML5, we can use attributes prefixed with `data-` to store information. We will use the `data-` attribute later.

Width from 520 to 699px - Apply an email icon to each link

When the browser's width is between 520 and 699px, we will apply an email icon to each email link:

Example 2

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul {
```

```
    list-style-type: none;
```

```
}
```

```
ul li a {
```

```
    color: green;
```

```
text-decoration: none;

padding: 3px;

display: block;

}
```

```
@media screen and (max-width: 699px) and (min-width: 520px) {

  ul li a {

    padding-left: 30px;

    background: url(email-icon.png) left center no-repeat;

  }

}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Resize the browser window to see the effect!</h1>
```

```
<ul>
```

```
  <li><a data-email="johndoe@example.com"
href="mailto:johndoe@example.com">John Doe</a></li>
```

```
  <li><a data-email="marymoe@example.com"
href="mailto:marymoe@example.com">Mary Moe</a></li>
```



```
<li><a data-email="amandapanda@example.com"
href="mailto:amandapanda@example.com">Amanda Panda</a></li>
</ul>
```

```
</body>
```

```
</html>
```

Width from 700 to 1000px - Preface the links with a text

When the browser's width is between from 700 to 1000px, we will preface each email link with the text "Email: ":

Example 3

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul {
```

```
    list-style-type: none;
```

```
}
```

```
ul li a {
```

```
    color: green;
```

```
    text-decoration: none;
```

```
padding: 3px;
display: block;
}
```

```
@media screen and (max-width: 699px) and (min-width: 520px) {
  ul li a {
    padding-left: 30px;
    background: url(email-icon.png) left center no-repeat;
  }
}
```

```
@media screen and (max-width: 1000px) and (min-width: 700px) {
  ul li a:before {
    content: "Email: ";
    font-style: italic;
    color: #666666;
  }
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Resize the browser window to see the effect!</h1>
```

```
<ul>
```

```
  <li><a data-email="johndoe@example.com"
href="mailto:johndoe@example.com">John Doe</a></li>
```

```
  <li><a data-email="marymoe@example.com"
href="mailto:marymoe@example.com">Mary Moe</a></li>
```

```
  <li><a data-email="amandapanda@example.com"
href="mailto:amandapanda@example.com">Amanda Panda</a></li>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Width above 1001px - Apply email address to links

When the browser's width is above 1001px, we will append the email address to the links.

We will use the value of the `data-` attribute to add the email address after the person's name:

Example 4

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
ul {  
    list-style-type: none;  
}
```

```
ul li a {  
    color: green;  
    text-decoration: none;  
    padding: 3px;  
    display: block;  
}
```

```
@media screen and (max-width: 699px) and (min-width: 520px) {  
    ul li a {  
        padding-left: 30px;  
        background: url(email-icon.png) left center no-repeat;  
    }  
}
```

```
@media screen and (max-width: 1000px) and (min-width: 700px) {  
    ul li a:before {  
        content: "Email: ";  
        font-style: italic;
```

```
        color: #666666;
    }
}
```

```
@media screen and (min-width: 1001px) {
    ul li a:after {
        content: " (" attr(data-email) ")";
        font-size: 12px;
        font-style: italic;
        color: #666666;
    }
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Resize the browser window to see the effect!</h1>
```

```
<ul>
```

```
    <li><a data-email="johndoe@example.com"
href="mailto:johndoe@example.com">John Doe</a></li>
```

```
<li><a data-email="marymoe@example.com"
href="mailto:marymoe@example.com">Mary Moe</a></li>

<li><a data-email="amandapanda@example.com"
href="mailto:amandapanda@example.com">Amanda Panda</a></li>

</ul>

</body>

</html>
```

Width above 1151px - Add icon as we used before

For browser widths above 1151px, we will again add the icon as we used before.

Here, we do not have to write an additional media query block, we can just append an additional media query to our already existing one using a comma (this will behave like an OR operator):

Example 5

```
<!DOCTYPE html>

<html>

<head>

<style>

ul {

    list-style-type: none;

}
```

```
ul li a {  
    color: green;  
    text-decoration: none;  
    padding: 3px;  
    display: block;  
}
```

```
@media screen and (max-width: 699px) and (min-width: 520px), (min-width:  
1151px) {  
    ul li a {  
        padding-left: 30px;  
        background: url(email-icon.png) left center no-repeat;  
    }  
}
```

```
@media screen and (max-width: 1000px) and (min-width: 700px) {  
    ul li a:before {  
        content: "Email: ";  
        font-style: italic;  
        color: #666666;  
    }  
}
```

```
@media screen and (min-width: 1001px) {
```

```
  ul li a:after {
```

```
    content: " (" attr(data-email) ")";
```

```
    font-size: 12px;
```

```
    font-style: italic;
```

```
    color: #666666;
```

```
  }
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Resize the browser window to see the effect!</h1>
```

```
<ul>
```

```
  <li><a data-email="johndoe@example.com"
href="mailto:johndoe@example.com">John Doe</a></li>
```

```
  <li><a data-email="marymoe@example.com"
href="mailto:marymoe@example.com">Mary Moe</a></li>
```

```
  <li><a data-email="amandapanda@example.com"
href="mailto:amandapanda@example.com">Amanda Panda</a></li>
```

```
</ul>
```


</body>

</html>